

LAPORAN TUGAS 4 II4021 KRIPTOGRAFI

Implementasi Shamir Secret Sharing pada Aplikasi Password Manager Terdistribusi



Dosen Pengampu: Dr. Ir. Rinaldi Munir, M.T.

Disusun oleh:

Kelompok 1

Nathaniel Jonathan Rusli (13523013)

Maheswara Bayu Kaindra (13523015)

Peter Wongsoredjo (13523039)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI.....	2
PERNYATAAN.....	3
BAB I LANDASAN TEORI.....	4
1.1 Shamir Secret Sharing.....	4
1.2 Advanced Encryption Standard (AES).....	5
1.2.1 Advanced Encryption Standard - Galois/Counter Mode (AES-GCM).....	6
1.3 Key Derivation Function (KDF).....	7
1.4 Cryptographically Secure Pseudorandom Number Generator (CSPRNG).....	8
1.5 Visual Cryptography (Kriptografi Visual).....	9
BAB II PERANCANGAN DAN IMPLEMENTASI.....	11
2.1 Client: Core Crypto Utilities.....	11
2.1.1 CSPRNG.....	11
2.1.2 Shamir Secret Sharing.....	11
2.1.3 Key Derivation Function.....	12
2.1.4 AES-GCM.....	12
2.2 Client: Local Storage, Manajemen Vault, Visual Cryptography.....	12
2.2.1 Struktur Penyimpanan Data Lokal.....	12
Dengan keterangan sebagai berikut.....	13
2.2.2 Alur Inisialisasi Vault dan Distribusi Shares.....	13
2.2.3 Mekanisme Akses dan Rekonstruksi Kunci.....	14
2.2.4 Manajemen Data Password.....	15
2.2.5 Implementasi Kriptografi Visual.....	16
2.3 Server: Docker, Database, and API Endpoints.....	17
2.3.1 Konfigurasi Docker File and Docker Compose Server.....	17
2.3.2 Konfigurasi Database SQLite Server.....	17
2.3.3 Konfigurasi API Routes dan Endpoints Server.....	18
BAB III PENGUJIAN DAN ANALISIS HASIL.....	20
3.1 Pengujian Pembuatan Vault.....	20
3.1.1 Inisialisasi Vault dan Pembangkitan Kunci.....	20
3.1.2 Tampilan Recovery Share.....	21
3.2 Pengujian Penyimpanan dan Perlindungan Local Share.....	22
3.2.1 Verifikasi Enkripsi State Lokal.....	22
3.2.2 Dekripsi dengan Kredensial Valid.....	22
3.2.3 Penolakan Kredensial yang Invalid.....	24
3.3 Pengujian Akses Normal.....	25
3.3.1 Rekonstruksi dan Akses Vault Valid.....	25
3.3.2 Kegagalan Dekripsi Akibat Share Invalid.....	26
3.4 Pengujian Penambahan Data Password.....	28
3.4.1 Penambahan dengan Input Manual.....	28

3.4.2 Penambahan dengan Pembangkitan Otomatis.....	29
3.4.3 Enkripsi Ulang Vault dan Pembaruan Backup Vault.....	31
3.5 Pengujian Pengubahan dan Penghapusan Data Password.....	33
3.5.1 Pengubahan Data Password.....	33
3.5.2 Penghapusan Data Password.....	35
3.5.3 Validasi Perubahan Hanya pada Mode Normal.....	37
3.6 Pengujian Penyimpanan Vault di Server.....	39
3.6.1 Vault Tersimpan sebagai BLOB.....	39
3.6.2 Validasi Penyimpanan Server.....	40
3.6.3 Validasi Penerimaan Server.....	41
3.7 Pengujian Mode Backup.....	43
3.7.1 Akses Vault secara Luring.....	43
3.7.2 Pembatasan Akses (Read-Only).....	45
3.8 Pengujian Kegagalan Pemulihan.....	47
3.8.1 Penggunaan Recovery Share yang Salah.....	47
3.8.2 Manipulasi Backup Vault.....	48
3.9 Pengujian Kriptografi Visual (Bonus).....	50
3.9.1 Pembangkitan dan Pembagian Gambar Share.....	50
3.9.2 Kombinasi Gambar Share.....	51
BAB IV KESIMPULAN.....	53
DAFTAR PUSTAKA.....	54
LAMPIRAN.....	55

PERNYATAAN

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (*raw output*) dari *Generative AI*, melainkan hasil pengembangan dan penulisan mandiri.

		
Nathaniel Jonathan Rusli	Maheswara Bayu Kandra	Peter Wongsoredjo

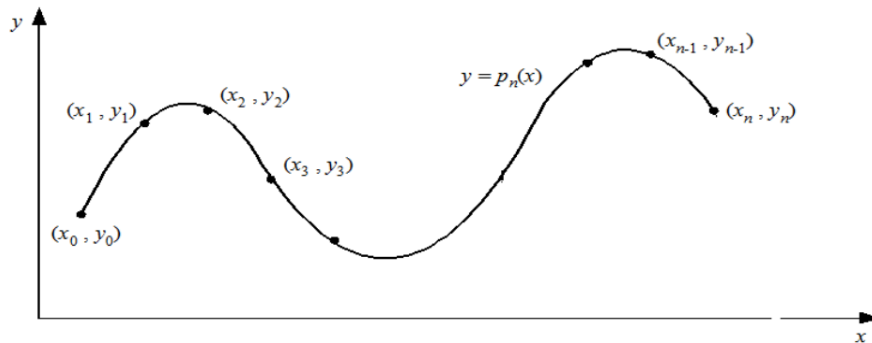
BAB I

LANDASAN TEORI

1.1 Shamir Secret Sharing

Menurut Munir (2026), Skema Pembagian Data Rahasia (*Secret Sharing Scheme*) adalah metode untuk membagi data atau informasi rahasia kepada beberapa partisipan. Salah satu algoritma yang paling dikenal adalah *Shamir threshold scheme*, di mana sebuah rahasia S dibagi kepada n partisipan sedemikian rupa sehingga sembarang himpunan bagian yang terdiri dari minimal t partisipan dapat merekonstruksi S . Sebaliknya, jika jumlah partisipan yang berkumpul kurang dari nilai ambang batas t ($S < t$), maka rahasia S tersebut sama sekali tidak dapat direkonstruksi.

Ide dasar dari skema penemuan Shamir ini bertumpu pada persoalan matematis interpolasi polinomial, di mana untuk membentuk sebuah polinomial berderajat $t - 1$ secara utuh, mutlak dibutuhkan setidaknya t buah titik koordinat.



Gambar 1.1.1 Grafik interpolasi polinomial Shamir Secret Sharing

Dalam mekanisme pembuatannya, sebuah rahasia S disembunyikan sebagai nilai intersep pada polinomial acak $f(x)$ dengan modulus p berupa sebuah bilangan prima yang ukurannya harus lebih besar dari nilai rahasia maupun jumlah partisipan.

$$f(x) \equiv S + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1} \pmod{p}$$

Pihak pen-distribusi (*dealer*) kemudian memberikan nilai evaluasi dari fungsi matematis tersebut sebagai titik koordinat (x_i, y_i) kepada masing-masing partisipan sebagai bagian *share* mereka. Ketika t orang partisipan menggabungkan *share* rahasia mereka, persamaan linier yang terbentuk dapat diselesaikan menggunakan teknik eliminasi *Gauss-Jordan* untuk menemukan kembali representasi utuh dari rahasia S .

1.2 Advanced Encryption Standard (AES)

Menurut Munir (2025), secara struktural AES tidak menggunakan arsitektur Feistel Network seperti pendahulunya, melainkan mengadopsi kerangka Substitution-Permutation Network (SPN) yang memproses data matriks secara dinamis melalui sejumlah iterasi putaran tertentu (10, 12, atau 14 putaran bergantung pada panjang kunci). Pada setiap putarannya, matriks data mengalami empat operasi transformasi matematis beruntun: SubBytes (substitusi non-linier menggunakan tabel S-box), ShiftRows (pergeseran sirkuler matriks baris), MixColumns (pencampuran data kolom menggunakan operasi polinomial medan berhingga Galois Field), dan AddRoundKey (operasi XOR dengan sub-kunci turunan dari kunci utama). Rangkaian proses difusi dan konfusi yang ekstensif ini menghasilkan ciperteks yang sangat acak, menjadikan AES sangat resisten terhadap metode peretasan kriptanalisis linear maupun eksploitasi serangan brute-force pada teknologi komputasi saat ini.

Tabel 1.2.1 Perbandingan Varian AES

Spesifikasi Varian AES	Ukuran Blok Data	Panjang Kunci Simetris	Jumlah Putaran Transformasi (Ronde)
AES-128	128-bit	128-bit	10 Putaran
AES-192	128-bit	192-bit	12 Putaran
AES-256	128-bit	256-bit	14 Putaran

Secara struktural, AES tidak menggunakan arsitektur Feistel Network seperti pendahulunya, melainkan mengadopsi kerangka Substitution-Permutation Network (SPN) yang memproses data matriks secara dinamis melalui sejumlah iterasi putaran tertentu (10, 12, atau 14 putaran bergantung pada panjang kunci). Pada setiap putarannya, matriks data mengalami empat operasi transformasi matematis beruntun: SubBytes (substitusi non-linier menggunakan tabel S-box), ShiftRows (pergeseran sirkuler matriks baris), MixColumns (pencampuran data kolom menggunakan operasi polinomial medan berhingga Galois Field), dan AddRoundKey (operasi XOR dengan sub-kunci turunan dari kunci utama). Rangkaian proses difusi dan konfusi yang

1.2.1 Advanced Encryption Standard - Galois/Counter Mode (AES-GCM)

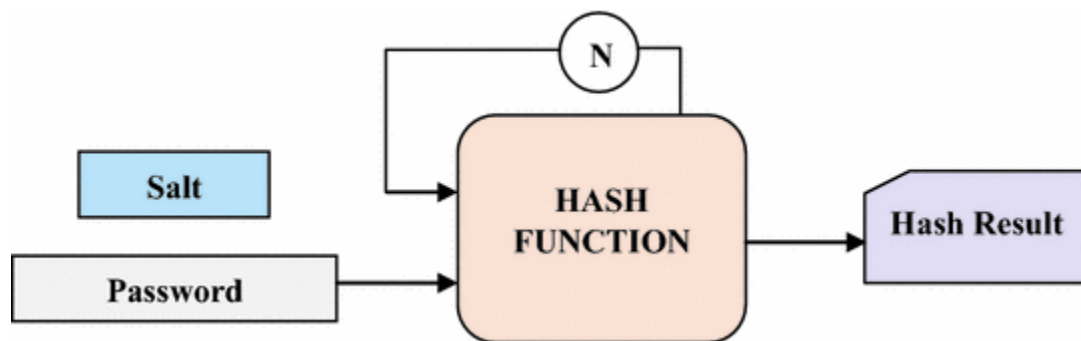
Gambar 1.2.2 AES-GCM

Sebagai solusi arsitektural yang lebih mutakhir, AES-GCM256 hadir dengan menggabungkan mode *Counter* (CTR) untuk proses enkripsi yang dapat dieksekusi secara paralel, serta operasi perkalian medan Galois (*Galois field*) untuk menghasilkan nilai autentikasi. Mode operasi ini tergolong ke dalam skema *Authenticated Encryption with Associated Data* (AEAD), yang berarti selain menjamin kerahasiaan (*confidentiality*) melalui penggunaan kunci simetris 256-bit yang sangat masif, algoritma ini juga secara simultan memverifikasi bahwa *ciphertext* tidak mengalami manipulasi (*tampering*) selama masa transmisi. Kombinasi antara efisiensi komputasi yang tinggi dan keamanan ganda (kerahasiaan serta integritas) menjadikan

AES-GCM256 sebagai standar operasi yang sangat direkomendasikan untuk sistem pertukaran pesan yang aman.

1.3 Key Derivation Function (KDF)

Key Derivation Function (KDF) merupakan algoritma kriptografi deterministik yang ditujukan secara spesifik untuk merekayasa dan mengonversi masukan dasar dengan entropi rendah, seperti *password*, menjadi sebuah kunci enkripsi yang tangguh. Fungsionalitas KDF menduduki esensi krusial sebagai fondasi lapisan mitigasi defensif terakhir ketika basis data kredensial sebuah sistem diretas, mencegah penyusup mengetahui kata sandi asli melalui *brute-force attack* maupun melalui strategi manipulasi kalkulasi dari tabel *rainbow table*.

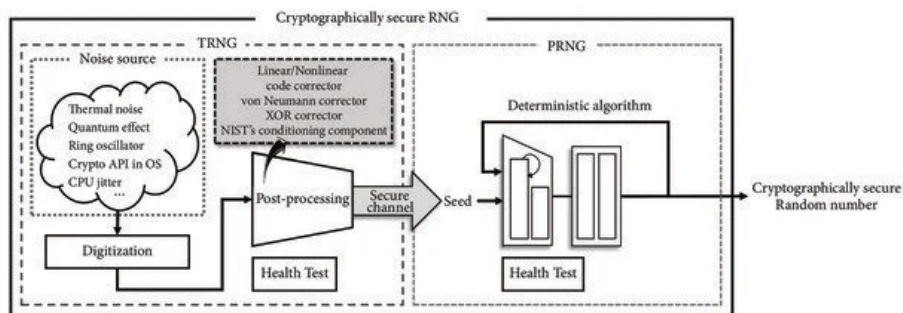


Gambar 1.3.1 Proses Algoritma *Key Derivation Function*

Mekanisme operasional algoritma KDF yang distandarisasi secara global (seperti implementasi PBKDF2) secara inheren mengeksplotasi fungsi pseudo-acak internal, layaknya modul algoritma *Keyed-Hash Message Authentication Code* (HMAC), yang dipacu untuk melakukan iterasi puluhan hingga ratusan ribu putaran siklus matematis. Eksekusi komputasi rekursif ini dikondisikan agar memakan banyak memori atau komputasi prosesor, sehingga mengakibatkan beban biaya kalkulasi yang eksponensial bagi penyerang. Di samping pertahanan iteratif, arsitektur KDF mewajibkan penambahan elemen *salt*, yakni parameter nilai acak berdimensi spesifik yang diinjeksikan secara dinamis bersama kata sandi pra pemrosesan. Intervensi entropi *salt* ini menjamin asuransi mutlak bahwa dua pengguna dengan kata sandi yang persis identik satu sama lain, menghasilkan derivasi *hash* akhir yang disalurkan ke dalam sistem penyimpanan *server* akan memiliki struktur sandi yang sama sekali berbeda dan tidak beririsan.

1.4 Cryptographically Secure Pseudorandom Number Generator (CSPRNG)

Cryptographically Secure Pseudorandom Number Generator (CSPRNG) merupakan utilitas pembangkit deret bilangan pseudo-acak yang didesain, ditinjau, dan dievaluasi secara komputasional agar mematuhi prasyarat ketat keamanan kriptografi modern. Dalam implementasi protokol kriptografi, penciptaan variabel sandi fundamental seperti nilai *nonce*, *salt*, *Initialization Vector* (IV), hingga penciptaan arsitektur pasangan kunci asimetris, mengharuskan entropi angka yang sama sekali tidak memiliki bias pola dan tidak bisa ditebak.

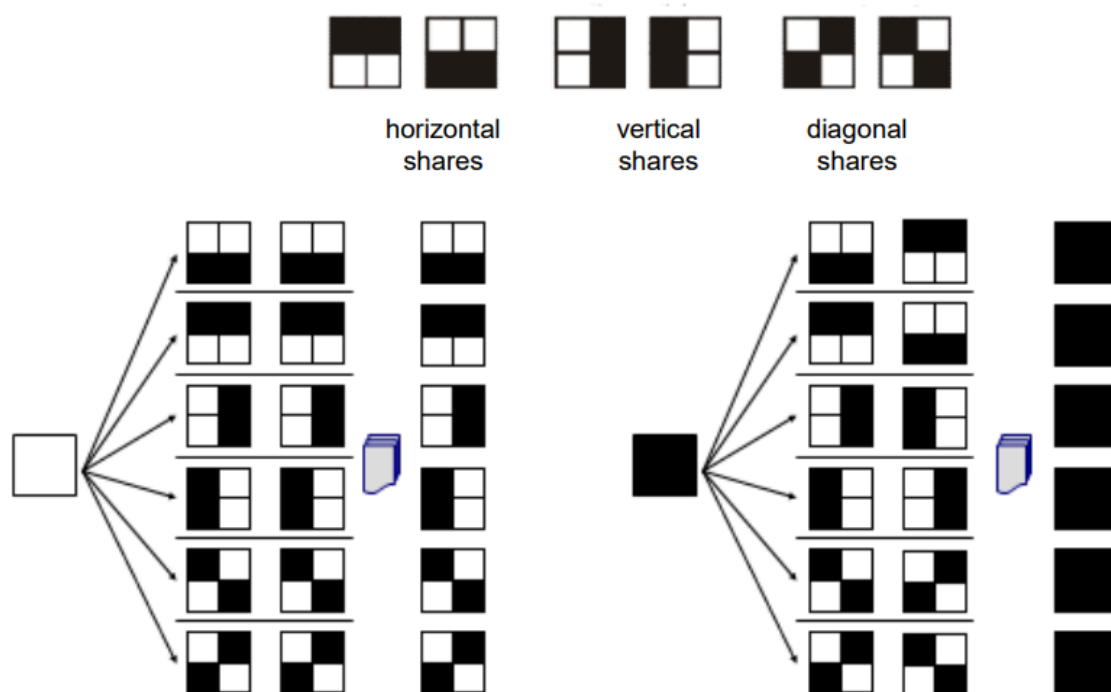


Gambar 1.4.1 Skema Konseptual CSPRNG

Perbedaan CSPRNG dengan rutinitas dari Pseudorandom Number Generator (PRNG) standar berada pada level resistensinya terhadap determinisme analitik empiris. Pada kerangka algoritma PRNG, seorang peretas yang entah bagaimana berhasil meretas akses menuju *seed* atau *internal state* akan mampu mensimulasikan kembali serangkaian nilai acak lanjutan dengan tingkat presisi prediksi yang akurat. Sebaliknya, protokol CSPRNG wajib dikonstruksikan dan dikalibrasi untuk selalu mendominasi kualifikasi uji statistik keras seperti *next-bit test*, menjamin klaim teoretis bahwa tidak terdapat entitas fungsi polinomial mana pun di bumi yang dapat menduga deret bit acak selanjutnya dengan kesuksesan probabilitas melewati nilai 50%, bahkan apabila peretas memonitor rekam jejak keluaran bit angka di tahap awal sandi. Properti ketidakpastian non-deterministik ini ditopang oleh modul permutasi fungsi *hash* (sesuai kepatuhan dokumentasi NIST SP 800-90A) yang memastikan perputaran siklus komputasinya hampir mustahil untuk didekripsi susunan awalnya.

1.5 Visual Cryptography (Kriptografi Visual)

Kriptografi visual merupakan sebuah teknik kriptografi yang mengenkripsi informasi visual dengan cara yang unik, sehingga proses dekripsi cukup dilakukan dengan mempersepsi informasi menggunakan penglihatan manusia tanpa perlu bantuan komputasi perangkat lunak. Menurut Munir (2026), dalam mekanisme dasar ini, sebuah gambar rahasia (*plain-image*) dipecah dan dibagi menjadi sejumlah gambar pecahan terpisah yang disebut *share* atau *shadow*. Setiap *share* secara individual hanya akan terlihat menyerupai kumpulan piksel acak yang tidak bermakna layaknya *noise*, sehingga tidak ada satupun *share* yang membocorkan secuil pun informasi dari gambar aslinya. Keunggulan paling esensialnya terletak pada sifat dekripsi komputasi yang ditiadakan (*fast decoding*), di mana partisipan hanya perlu menumpuk *share* secara presisi untuk langsung membaca pesan rahasia tersebut.



Gambar 1.5.. Ilustrasi skema (2,2) kriptografi visual

Sebagai implementasi konkret, skema (2, 2) untuk citra biner mendistribusikan satu gambar rahasia secara utuh ke dalam dua buah *share*, yang mana keduanya mutlak diperlukan bersamaan untuk merekonstruksi citra. Algoritma enkripsi ini beroperasi dengan membagi setiap satu piksel warna dari gambar asli menjadi sejumlah sub-piksel di dalam *share* pecahannya.

Apabila piksel asli berwarna putih, algoritma akan mengambil susunan matriks acak dari himpunan permutasi C_0 , sedangkan untuk piksel berwarna hitam, sistem akan menyeleksi matriks dari himpunan C_1 . Oleh karena mekanisme penumpukan transparansi secara logis bertindak layaknya fungsi operasi komputasional OR, tumpukan matriks dari kedua share tersebut akan menghasilkan blok sub-piksel yang secara visual dapat dikontraskan kembali sebagai warna hitam atau putih.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1 *Client: Core Crypto Utilities*

2.1.1 CSPRNG

Implementasi *Cryptographically Secure Pseudorandom Number Generator* (CSPRNG) digunakan secara krusial di beberapa titik sistem, seperti pembuatan *master key*, pembangkitan *salt* KDF, dan pembuatan *nonce* untuk AES-GCM. Modul ini menggunakan *library* bawaan Python yaitu *secrets*, yang dijamin aman secara kriptografis karena mengambil entropi langsung dari sistem operasi. Untuk fitur pembuatan kata sandi otomatis, CSPRNG digunakan untuk memilih karakter secara acak dari kamus yang berisi huruf besar, huruf kecil, angka, dan simbol. Pemetaan dilakukan dengan memanggil fungsi utilitas dari pustaka *secrets* untuk memastikan distribusi probabilitas karakter merata dan kebal terhadap prediksi.

2.1.2 *Shamir Secret Sharing*

Implementasi *Shamir Secret Sharing* terdapat pada file `src/client/crypto/sss.py`. Kunci berukuran 16-byte (128-bit) yang dihasilkan oleh CSPRNG dibagi menggunakan algoritma Shamir Secret Sharing dengan skema ambang batas (2, 3). Implementasi dilakukan memanfaatkan modul **Shamir** dari **pycryptodome** yang melakukan operasi matematika polinomial di atas *Galois Field* $GF(2^8)$, memastikan operasi pada *array byte* dapat dilakukan tanpa memicu komputasi *floating-point*.

Output dari fungsi pembagian kunci diubah formatnya menjadi bentuk *string* dengan pola `index-hex_string` (contoh: 1-a4f...) agar *Recovery Share* dapat ditampilkan dalam format yang mudah disalin atau dicatat oleh pengguna. Sistem juga akan melakukan *parsing* pemisahan string tersebut kembali menjadi indeks dan *byte array* sebelum digabungkan. Jika dua *share* yang dimasukkan valid, *master key* asli akan berhasil dipulihkan secara matematis.

2.1.3 Key Derivation Function

Local share wajib dilindungi dengan kunci turunan dari *master password* yang diketikkan oleh pengguna. Sistem dirancang menggunakan PBKDF2 HMAC dengan fungsi *hash* SHA-256. Untuk menahan *brute-force* dan *dictionary attack*, parameter iterasi (*cost*) ditetapkan pada 600.000 iterasi, menyesuaikan dengan standar rekomendasi keamanan OWASP tahun 2023. Untuk mencegah *rainbow table*, Setiap pembuatan *vault* baru akan membangkitkan 16-byte *salt* acak yang disimpan secara lokal. *Salt* ini digabungkan dengan *password* saat proses derivasi untuk memastikan *password* yang sama akan tetap menghasilkan kunci turunan 16-byte (128-bit) yang sama sekali berbeda pada *vault* pengguna yang berbeda. Keseluruhan modul terdapat di file `src/client/crypto/kdf.py`

2.1.4 AES-GCM

Algoritma AES-GCM dipilih untuk memberikan kerahasiaan sekaligus menjamin integritas data. Setiap operasi enkripsi (baik untuk *vault* utama, *backup vault* lokal, maupun *local share*) akan selalu membangkitkan 12-byte *nonce* acak baru melalui CSPRNG. *Nonce* ini kemudian dikembalikan bersamaan dengan *ciphertext* untuk disimpan di *server* atau basis data lokal. Untuk memvalidasi integritas, digunakan MAC. Pada saat proses dekripsi, fungsi dirancang untuk menangkap *exception* `InvalidTag`. Apabila *server* memodifikasi isi *vault*, atau pengguna memasukkan *recovery share* yang keliru (menyebabkan *master key* hasil rekonstruksi salah), dekripsi tidak akan berlangsung, dan melempar `DecryptionError` untuk menghentikan alur aplikasi dengan aman.

2.2 Client: Local Storage, Manajemen Vault, Visual Cryptography

2.2.1 Struktur Penyimpanan Data Lokal

Penyimpanan *state* di sisi client diimplementasikan menggunakan file JSON bernama `client_data.json`. Untuk menjaga integritas data saat disimpan dalam format teks, seluruh *byte array* (*ciphertext*, *nonce*, dan *salt*) dikodekan terlebih dahulu menjadi suatu string heksadesimal (diimplementasikan pada `storage.py`). Dengan ini, klien tidak menyimpan kredensial dalam bentuk *plaintext* (*zero-knowledge*).

Tabel 2.2.1.1 Skema Tabel vault

Nama Field	Bentuk Isi
kdf_salt	<hex_string>
local_share_cipher	<hex_string>
local_share_nonce	<hex_string>
backup_vault_cipher	<hex_string>
backup_vault_nonce	<hex_string>

Dengan keterangan sebagai berikut,

1. kdf_salt: Parameter *salt* acak yang digunakan oleh Key Derivation Function (KDF) untuk menjamin penurunan kunci yang deterministik dari password yang dimasukkan pengguna.
2. local_share_cipher: satu dari tiga bagian *master key* hasil Shamir Secret Sharing yang disimpan dalam keadaan terenkripsi. Data ini dikunci menggunakan *authenticated encryption* dengan kunci turunan dari *master password*.
3. local_share_nonce: nilai acak sekali pakai untuk dekripsi AES-128-GCM terhadap local_share_cipher saat pengguna melakukan autentikasi.
4. backup_vault_cipher: salinan isi *vault* pengguna yang dienkripsi secara lokal.
5. backup_vault_nonce: *nonce* yang dipasangkan dengan backup_vault_cipher untuk mendekripsi *backup vault*.

2.2.2 Alur Inisialisasi Vault dan Distribusi Shares

Pada proses inisialisasi *vault*, siklus hidup kunci dimulai pada memori *client*, di mana *master key* dibangkitkan, dipecah, dan didistribusikan ke *client*, *server*, dan pengguna. Proses inisialisasi akun dan pembagian komponen kriptografis dilakukan di sisi klien dengan tahapan sebagai berikut (vault_manager.py dan visual_crypto.py).

1. Klien membangkitkan *master_key* secara acak menggunakan `sss.generate_master_key()`.

2. `master_key` dipecah menjadi 3 bagian menggunakan Shamir Secret Sharing dengan batas (2,3) dengan `sss.split_key()`, menghasilkan `local_share` (indeks 0), `server_share` (index 1), dan `recovery_share` (index 2).
3. Klien membangkitkan *salt* dengan `kdf.generate_salt()` dan menurunkan `derived_key` dan `master_password` dengan `kdf.derive_key(master_password, salt)`.
4. String `local_share` dikodekan ke UTF-8 dan dienkripsi dengan AES-128-GCM menggunakan `derived_key`. Hasilnya (`local_share_cipher` dan `local_share_nonce`) disimpan langsung ke `client_data.json` bersama *salt* dengan `storage.save_client_config()`.
5. Struktur data *vault* (kosong) diinisialisasi dalam format JSON biner, lalu dienkripsi dengan AES-128-GCM menggunakan `master_key` menghasilkan `vault_cipher` dan `vault_nonce`. Klien menyimpan parameter ini dengan `storage.save_backup_vault()`.
6. Klien menyusun *payload* JSON berisi `username`, `server_share`, `vault_ciphertext`, dan `vault_nonce` dan mengirimnya ke server via HTTP POST ke `/api/vault/init`.
7. Setelah menerima respons HTTP 200 dari server, `main.py` mengirim `recovery_share` ke fungsi `generate_visual_shares()`. Kunci diubah menjadi QR Code lalu dipecah menjadi dua berkas gambar (`share1.png` dan `share2.png`).

2.2.3 Mekanisme Akses dan Rekonstruksi Kunci

Terdapat dua jalur eksekusi untuk membuka *vault* yang bergantung pada kesediaan server. Kedua mode mengharuskan autentikasi awal menggunakan *master password* untuk mendekripsi kunci lokal.

2.2.3.1 Mode Akses Normal

Diimplementasikan melalui fungsi `open_vault_normal()` (`vault_manager.py`), mode ini menggunakan kombinasi *share* lokal dan server.

1. Klien memuat `kdf_salt`, `local_share_cipher`, dan `local_share_nonce` dari `client_data.json` melalui modul `storage`.
2. Klien menurunkan `derived_key` dari input `master_password` dan `kdf_salt` menggunakan KDF. Kunci ini digunakan untuk mendekripsi `local_share_cipher` dengan fungsi `aes_gcm.decrypt_data()`, menghasilkan string `local_share`.

3. Klien mengeksekusi HTTP GET ke `/api/vault/{username}` untuk mengunduh `server_share`, `vault_ciphertext`, dan `vault_nonce`.
4. Fungsi `sss.reconstruct_key(local_share, server_share)` dijalankan untuk mendapatkan `master_key` di memori klien.
5. `master_key` digunakan untuk mendekripsi `vault_ciphertext` beserta `vault_nonce` menggunakan AES-128-GCM. *Byte array* yang dihasilkan diurai menjadi *dictionary*. Akses manipulasi (tambah, ubah, hapus) diizinkan.

2.2.3.2 Mode Akses *Backup*

Diimplementasikan melalui fungsi `open_vault_backup()`, mode ini berjalan dengan data lokal dan *input* pengguna.

1. Pengguna memasukkan *master password* dan *recovery share* melalui CLI.
2. Klien memuat konfigurasi *local share* (`kdf_salt`, `local_share_cipher`, `local_share_nonce`) beserta salinan *backup vault* dari `client_data.json`.
3. Klien mengeksekusi KDF menggunakan *master password* dan `kdf_salt`, menghasilkan `derived_key`.
4. Klien menggunakan `derived_key` untuk mendekripsi `local_shared_cipher` dengan AES-128-GCM, sehingga *local share* berhasil diekstrak di memori.
5. Klien memulihkan *master key* dengan Shamir Secret Sharing.
6. Klien menggunakan *master key* untuk mendekripsi data `backup_vault_cipher`.
7. Klien dapat mengakses vault dalam mode *read-only*.

2.2.4 Manajemen Data Password

Penambahan, pengubahan, dan penghapusan data *password* dieksekusi terhadap struktur data vault (*dictionary* JSON) di memori klien, dengan urutan sebagai berikut.

1. Klien memperbarui `vault_data` di memori berdasarkan input dari pengguna (menambah entri baru, mengubah *username/password*, atau menghapus kunci *dictionary* layanan).
2. Setelah terjadi perubahan, seluruh isi struktur *dictionary* dienkripsi ulang sebagai satu kesatuan (*ciphertext* BLOB) menggunakan AES-128-GCM dengan `master_key` dan `nonce` yang baru dibangkitkan.

3. Klien memanggil `storage.save_backup_vault()` untuk menyimpan `vault_ciphertext` dan `vault_nonce` yang baru untuk memperbarui kondisi *backup vault*.
4. Klien mengirimkan *payload* berisi `vault_ciphertext` dan `vault_nonce` baru ke server melalui HTTP PUT. Server menerima pembaruan ini dan menyimpannya ke basis data SQLite untuk menimpa versi vault sebelumnya pada mode normal.

2.2.5 Implementasi Kriptografi Visual

Pada implementasi ini, reks *recovery share* dikonversi menjadi gambar QR Code yang dipecah menjadi dua *share* berupa gambar dengan mekanisme sebagai berikut.

1. Teks *recovery share* dikonversi menjadi QR Code monokrom (hitam-putih) menggunakan modul `qrcode` dengan tingkat koreksi galat (*error correction*) L (Low).
2. Algoritma menggunakan modul `Pillow (PIL)` untuk membuat kanvas *share* baru dengan dimensi 2×2 kali lebih besar dari dimensi QR Code asli. Setiap 1 piksel pada matriks sumber akan diekspansi menjadi 1 blok matriks berukuran 2×2 piksel pada gambar hasil pembagian.
3. Klien mendefinisikan enam pola matriks 2×2 yang memuat rasio warna hitam dan putih secara seimbang (2 piksel hitam, 2 piksel putih). Pola-pola ini merepresentasikan susunan diagonal, horizontal, dan vertikal.
4. Algoritma *splitting*:
 - a. Untuk setiap piksel, klien secara acak memilih satu pola dari enam pola dasar yang ada, dan menetapkan sebagai blok matriks pada Share 1.
 - b. Jika piksel berwarna putih (nilai 255): Blok matriks pada Share 2 diatur secara identik dengan pola blok pada Share 1. Jika piksel sumber berwarna hitam (nilai 0): Blok matriks pada Share 2 diatur menjadi kebalikan (*inverse*) dari pola blok pada Share 1 (nilai piksel dikurangi dari 255).
5. Untuk memverifikasi *shares*, sistem membangkitkan gambar simulasi penggabungan dengan melakukan komparasi menggunakan fungsi `min()` terhadap nilai piksel Share 1 dan Share 2. Operasi ini sama dengan logika AND pada gambar asli (piksel hitam bernilai 0 akan mendominasi piksel putih bernilai 255).

Tabel 2.2.5.1 Tabel Kebenaran Pemetaan Piksel

Warna Piksel QR Code Asli	Blok Matriks <i>Share 1</i>	Blok Matriks <i>Share 2</i>	Hasil Penggabungan
Putih (Latar)	Pola Acak (50% Hitam, 50% Putih)	Sama dengan <i>Share 1</i>	Abu-Abu (50% hitam, 50% putih)
Hitam (Data)	Pola Acak (50% Hitam, 50% Putih)	Kebalikan <i>Share 1</i>	Hitam

2.3 Server: Docker, Database, and API Endpoints

2.3.1 Konfigurasi Docker File and Docker Compose Server

Docker File (/src/server/Dockerfile) dan Docker Compose (./docker-compose.yml) digunakan untuk memudahkan proses *setup* bagian aplikasi *server*. Docker File dengan *base image* python:3.12-slim juga diisi dengan instruksi instalasi seluruh *dependency* yang ada pada file requirements.txt (/src/server/requirements.txt) dan juga instruksi *command* untuk menjalankan server uvicorn sebagai penunjang seluruh *endpoints* Fast API dari sisi *server*. Di sisi lain, Docker Compose yang akan dijalankan dari *root* monorepo *client-server* ini berisi definisi atas *service* vault-server yang dijalankan dengan *build context* berupa Docker File sebelumnya dan juga deklarasi lainnya termasuk *port mapping*, *mounting volumes*, dan juga injeksi *environment variable*.

2.3.2 Konfigurasi Database SQLite Server

Proses dan kode *setup* basis data SQLite berpusat pada *file* src/server/db/database.py dengan fungsi utama *init_db()* dan juga fungsi pembantu *get_db_connection()*. Karena *project* ini ditulis dalam bahasa Python, cukup mengimpor library *sqlite3* yang merupakan *native Python library* dan menggunakan metode *connect* serta *cursor* untuk mengeksekusi *parameterized query* SQL yang diinginkan. Fungsi *init_db()* merupakan fungsi yang akan membuat tabel vault dengan definisi skema pada **Tabel 2.3.2.1** dan diharapkan proses inisialisasi tabel ini dieksekusi jika tabel belum terbentuk pada *DB_PATH*. Fungsi *get_db_connection* merupakan fungsi pembantu untuk memulai dan *return* objek koneksi ke basis data SQLite sesuai *DB_PATH*. Fungsi pembantu ini digunakan pada fungsi *init_db()* dan juga deklarasi seluruh *server route endpoints*.

Tabel 2.3.2.1 Skema Tabel vault

Nama <i>Field</i>	Tipe	<i>Constraints</i>
username	TEXT	Primary Key
server_share	TEXT	Not Null
vault_ciphertext	BLOB	Not Null
vault_nonce	BLOB	Not Null

2.3.3 Konfigurasi API *Routes* dan *Endpoints Server*

Seluruh API *routes* dan *endpoints* diimplementasikan dengan menggunakan Fast API dengan 3 *file* utama: 1) `src/server/main.py` untuk deklarasi Fast API *application* dan integrasi *router* serta deklarasi lifespan yaitu parameter aplikasi Fast API yang di sini digunakan untuk memastikan basis data telah dibuat dan diinisialisasi bila belum, 2) `src/server/api/schemas.py` yang berisi skema Pydantic guna validasi *request* dan *response format*, dan 3) `src/server/api/routes.py` yang berisi seluruh deklarasi *server endpoints*. Berikut merupakan seluruh skema Pydantic yang digunakan:

Tabel 2.3.3.1 Skema Pydantic

```

class VaultInitRequest(BaseModel):
    username: str
    server_share: str
    vault_ciphertext: str
    vault_nonce: str

class VaultUpdateRequest(BaseModel):
    vault_ciphertext: str
    vault_nonce: str

class VaultResponse(BaseModel):
    username: str
    server_share: str
    vault_ciphertext: str
    vault_nonce: str

```

Terdapat juga definisi *endpoint* yang didefinisikan pada sisi *server* seperti berikut:

Tabel 2.3.3.2 Definisi *Server Endpoints*

<i>Route / Endpoint</i>	<i>Method</i>	<i>Deskripsi</i>
/api/vault/init	POST	Menerima dan menyimpan data <i>vault</i> baru dari klien (mencakup identitas pengguna, <i>server share</i> , <i>ciphertext vault</i> awal, dan <i>nonce</i>).
/api/vault/{username}	GET	Mengirimkan kembali <i>server share</i> , <i>ciphertext vault</i> , dan <i>nonce</i> kepada klien untuk keperluan rekonstruksi <i>master key</i> dan dekripsi pada Mode Normal.
/api/vault/{username}	PUT	Menerima pembaruan data <i>ciphertext vault</i> dan <i>nonce</i> baru dari klien untuk menimpa data lama di SQLite, yang dipicu setelah proses penambahan, pengubahan, atau penghapusan data <i>password</i> di sisi klien.

BAB III

PENGUJIAN DAN ANALISIS HASIL

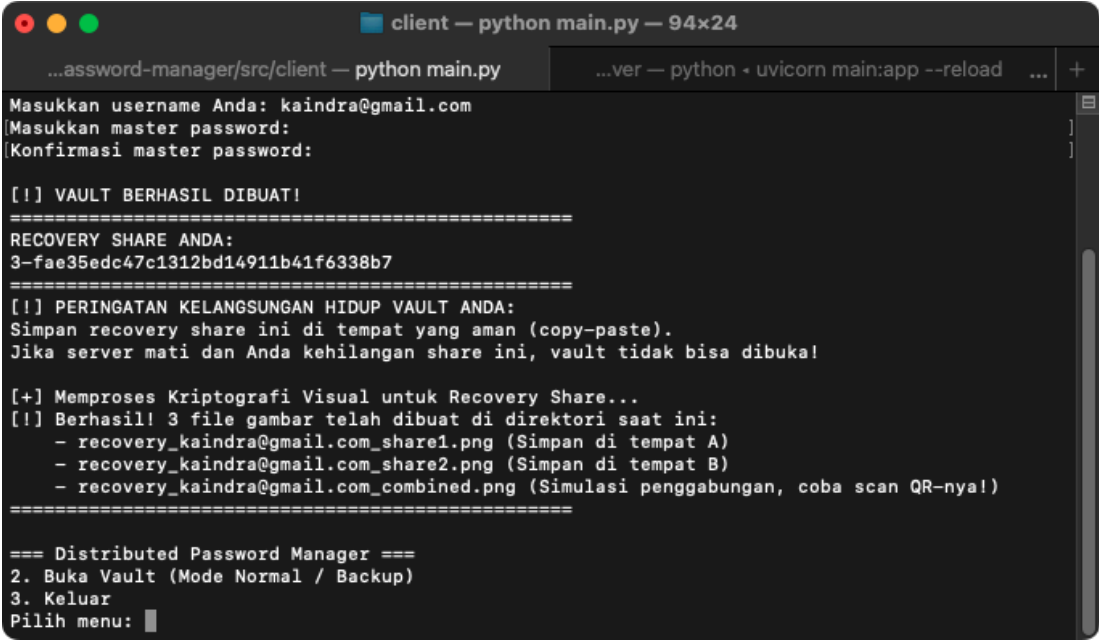
3.1 Pengujian Pembuatan *Vault*

3.1.1 Inisialisasi *Vault* dan Pembangkitan Kunci

Komponen	Keterangan
Tujuan Uji	Memastikan <i>master key</i> berhasil dibangkitkan, dibagi menggunakan Shamir Secret Sharing dengan skema (2,3), dan data <i>vault</i> awal berhasil dienkripsi.
Input	<i>username</i> dan <i>master password</i>
Prosedur	1. Pilih opsi “Buka Vault Baru”. 2. Masukkan data input yang diminta.
Output yang Diharapkan	Sistem berhasil membangkitkan <i>master key</i> , mengenkripsi <i>vault</i> kosong, dan membagi <i>master key</i> menjadi <i>local share</i> , <i>server share</i> , dan <i>recovery share</i> .

a. Hasil

Tampilan Terminal Client



```
client — python main.py — 94x24
...assword-manager/src/client — python main.py
Masukkan username Anda: kaindra@gmail.com
Masukkan master password:
Konfirmasi master password:

[!] VAULT BERHASIL DIBUAT!
=====
RECOVERY SHARE ANDA:
3-fae35edc47c1312bd14911b41f6338b7
=====
[!] PERINGATAN KELANGSUNGAN HIDUP VAULT ANDA:
Simpan recovery share ini di tempat yang aman (copy-paste).
Jika server mati dan Anda kehilangan share ini, vault tidak bisa dibuka!

[+] Memproses Kriptografi Visual untuk Recovery Share...
[!] Berhasil! 3 file gambar telah dibuat di direktori saat ini:
- recovery_kaindra@gmail.com_share1.png (Simpan di tempat A)
- recovery_kaindra@gmail.com_share2.png (Simpan di tempat B)
- recovery_kaindra@gmail.com_combined.png (Simulasi penggabungan, coba scan QR-nya!)
=====

=== Distributed Password Manager ===
2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: █
```

Gambar 3.1.1 Pembuatan akun (*vault*) berhasil

b. Analisis

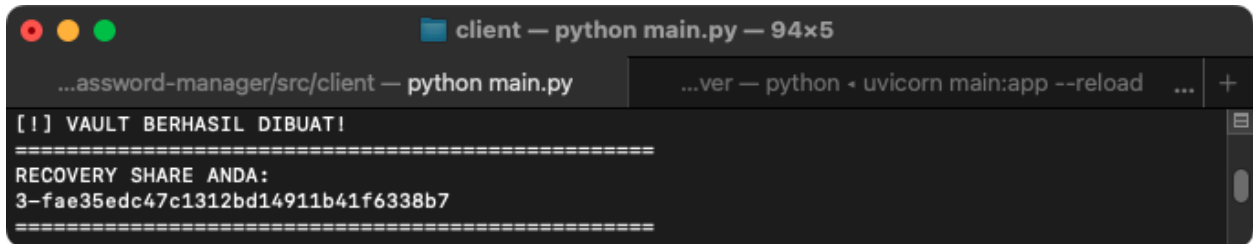
Pengujian inisialisasi *vault* dan pembangkitan kunci berhasil. Input kata sandi pengguna memicu `sss.generate_master_key()` untuk menghasilkan kunci biner acak, yang kemudian langsung diteruskan ke fungsi `sss.split_key()` untuk dipecah menjadi tiga buah *share* menggunakan skema (2, 3). Pada saat yang sama, *master key* digunakan oleh fungsi `aes_gcm.encrypt_data()` untuk mengenkripsi objek *dictionary* kosong sebelum akhirnya dihapus dari siklus eksekusi.

3.1.2 Tampilan *Recovery Share*

Komponen	Keterangan
Tujuan Uji	Memverifikasi format dan kemunculan <i>recovery share</i> kepada pengguna.
Input	Melanjutkan alur eksekusi dari test case 3.1.1.
Prosedur	Amati <i>output</i> pada terminal CLI sesaat setelah inisialisasi akun dinyatakan berhasil.
Output yang Diharapkan	<i>Recovery share</i> hanya ditampilkan dalam format teks yang secara jelas memuat koordinat dan nilai <i>share</i> agar dapat disalin pengguna.

a. Hasil

Tampilan Terminal Client



```

client — python main.py — 94x5
...assword-manager/src/client — python main.py  ...ver — python - uvicorn main:app --reload ... +
[!] VAULT BERHASIL DIBUAT!
=====
RECOVERY SHARE ANDA:
3-fae35edc47c1312bd14911b41f6338b7
=====

```

Gambar 3.1.2 *Recovery share* yang terbentuk saat *vault* terbentuk

b. Analisis

Fungsionalitas ini terbukti berhasil. Setelah data dienkripsi dan *server share* dikirim dengan HTTP POST, fungsi `init_vault` mengembalikan *string* `recovery_share` ke *thread* utama di `main.py`. CLI menangkap variabel tersebut dan melakukan *print* ke tampilan pengguna. Di luar itu, *recovery share* tidak pernah ditampilkan lagi.

3.2 Pengujian Penyimpanan dan Perlindungan *Local Share*

3.2.1 Verifikasi Enkripsi *State* Lokal

Komponen	Keterangan
Tujuan Uji	Memastikan <i>local share</i> tidak diekspos dalam format aslinya pada sistem berkas pengguna.
Input	client_data.json
Prosedur	Buka <i>file</i> client_data.json menggunakan editor teks dan amati nilai yang tersimpan.
Output yang Diharapkan	<i>Local share</i> tidak disimpan dalam bentuk asli, tetapi dienkripsi menggunakan kunci turunan dari <i>master password</i> dan disimpan dalam format heksadesimal.

a. Hasil

Tampilan Terminal Client

```
src > client > {} client_data.json > ...
1  {
2      "kdf_salt": "cc5bc13240fa47585763469ad4589db8",
3      "local_share_cipher": "99f9b9b0e098caf2e6accba123bccbe5863fce5ae37afe178576b422a133b9fc6b83db3dd95d300cabd",
4      "local_share_nonce": "d5272acc3d10cb2ea811f4b3",
5      "backup_vault_cipher": "294390733fdcfb478ad4452b550bc6144391",
6      "backup_vault_nonce": "3ab551b5b29d06f88f307c2e"
7  }
```

Gambar 3.2.1 Isi dokumen client_data.json

b. Analisis

Verifikasi enkripsi *local state* berhasil. Sesaat setelah pembagian kunci, nilai *local_share* langsung diteruskan ke fungsi `aes_gcm.encrypt_data()` untuk dienkripsi menggunakan kunci simetris hasil `output kdf.derive_key()`. Hasilnya yang berupa *ciphertext* dan *nonce* dikirim ke fungsi `storage.save_client_config()`, data tersebut dikonversi menjadi format string heksadesimal sebelum ditulis ke `client_data.json`.

3.2.2 Dekripsi dengan Kredensial Valid

Komponen	Keterangan
Tujuan Uji	Memastikan fungsionalitas KDF dan dekripsi berjalan sukses saat diberikan input yang valid.

Komponen	Keterangan
Input	master_password yang benar.
Prosedur	Jalankan klien, pilih opsi masuk, lalu masukkan master_password yang benar sesuai inisialisasi akun.
Output yang Diharapkan	Sistem mengonfirmasi otorisasi valid dan <i>local share</i> berhasil didekripsi ke dalam memori.

a. Hasil

Tampilan Terminal Client

```

client — python main.py — 94x22
...assword-manager/src/client — python main.py
...ver — python - uvicorn main:app --reload

=== Distributed Password Manager ===
2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: 2

[+] Mengakses vault...
Pilih mode akses:
1. Mode Normal (Koneksi ke Server)
2. Mode Backup (Darurat / Offline)
Masukkan pilihan (1/2): 1
Masukkan username Anda: kaindra@gmail.com
[Masukkan master password:

[+] BERHASIL: Otorisasi valid. Vault dibuka (Mode Normal).

=== Vault Menu (kaindra@gmail.com) [MODE NORMAL] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5):

```

Gambar 3.2.2 Login ulang dengan kredensial yang valid

b. Analisis

Pengujian dekripsi dengan kredensial valid berhasil. Fungsi `open_vault_normal` berhasil merekonstruksi kunci dengan input *master password* yang benar. Eksekusi dimulai ketika modul klien memanggil `storage.get_item()` untuk memuat mendapatkan `kdf_salt`, `local_share_cipher`, dan `local_share_nonce` dari penyimpanan lokal dan mengubahnya kembali menjadi format biner. Variabel *password* pengguna dan *salt* diteruskan ke `kdf.derive_key()` untuk mereproduksi kunci turunan yang sama persis dengan kunci saat fase inisialisasi. Kunci valid tersebut kemudian dimasukkan ke fungsi `aes_gcm.decrypt_data()`, yang secara artinya berhasil melewati

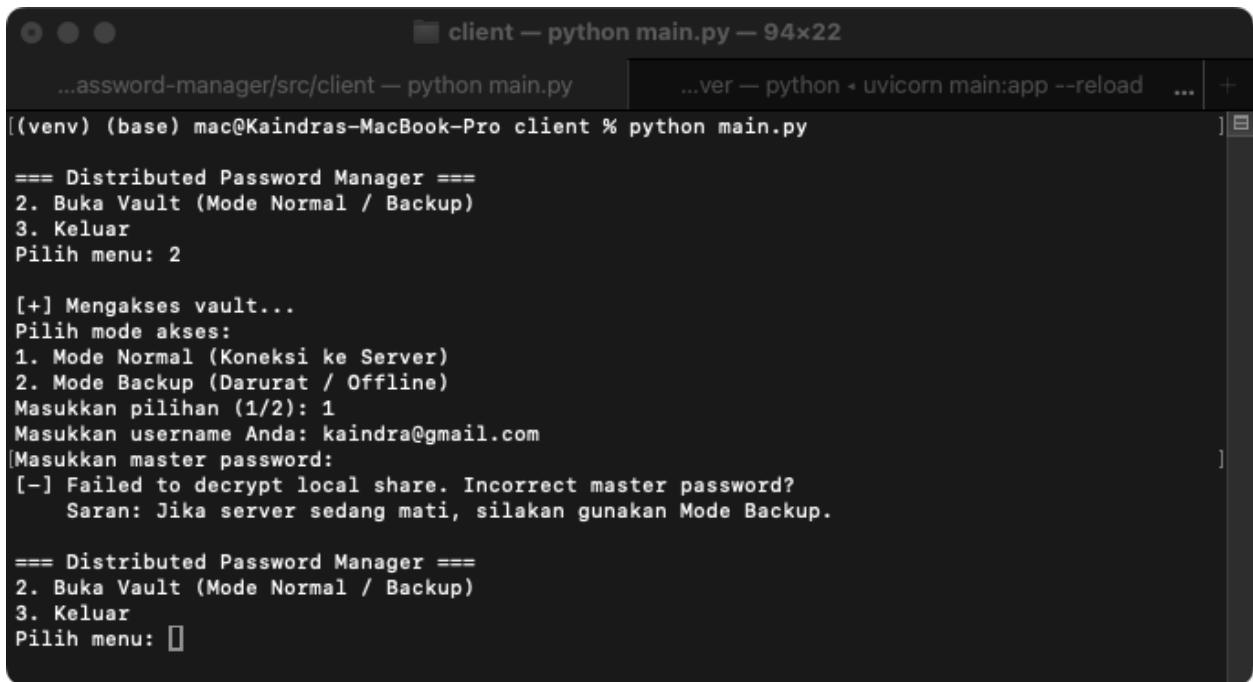
pengecekan tag autentikasi GCM, sehingga operasi dekripsi berhasil dijalankan dan string `local_share` asli berhasil diekstrak ke dalam memori klien.

3.2.3 Penolakan Kredensial yang *Invalid*

Komponen	Keterangan
Tujuan Uji	Memastikan penolakan akses apabila kunci turunan KDF yang dihasilkan berbeda.
Input	<code>master_password</code> yang salah.
Prosedur	Jalankan klien, pilih opsi masuk, lalu masukkan sembarang teks pada <i>prompt password</i> .
Output yang Diharapkan	Proses dekripsi melempar <i>exception</i> akibat ketidakcocokan autentikasi, sehingga <i>local share</i> gagal didekripsi dan akses ditolak.

a. Hasil

Tampilan Terminal Client



```
client — python main.py — 94x22
...assword-manager/src/client — python main.py
...ver — python • uvicorn main:app --reload ... +
[(venv) (base) mac@Kaindras-MacBook-Pro client % python main.py]
=== Distributed Password Manager ===
2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: 2

[+] Mengakses vault...
Pilih mode akses:
1. Mode Normal (Koneksi ke Server)
2. Mode Backup (Darurat / Offline)
Masukkan pilihan (1/2): 1
Masukkan username Anda: kaindra@gmail.com
[Masukkan master password:
[-] Failed to decrypt local share. Incorrect master password?
    Saran: Jika server sedang mati, silakan gunakan Mode Backup.

=== Distributed Password Manager ===
2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: ]
```

Gambar 3.2.3 Login gagal karena kredensial *invalid*

b. Analisis

Kredensial yang *invalid* berhasil memicu penolakan akses. Ketika pengguna memasukkan *password* atau *username* yang salah, pemanggilan `kdf.derive_key()` memproduksi `derived_key` biner yang berbeda dari kunci asli saat inisialisasi. Saat kunci yang salah ini diteruskan ke `aes_gcm.decrypt_data()` untuk membuka `local_share_cipher`, algoritma AES-128-GCM langsung gagal memverifikasi tag autentikasi bawaannya. Hal ini menyebabkan sistem melempar *exception*. *Exception* tersebut ditangkap, memaksa `local_share` tetap kosong, dan mengembalikan CLI dengan memberikan pesan penolakan akses kepada pengguna.

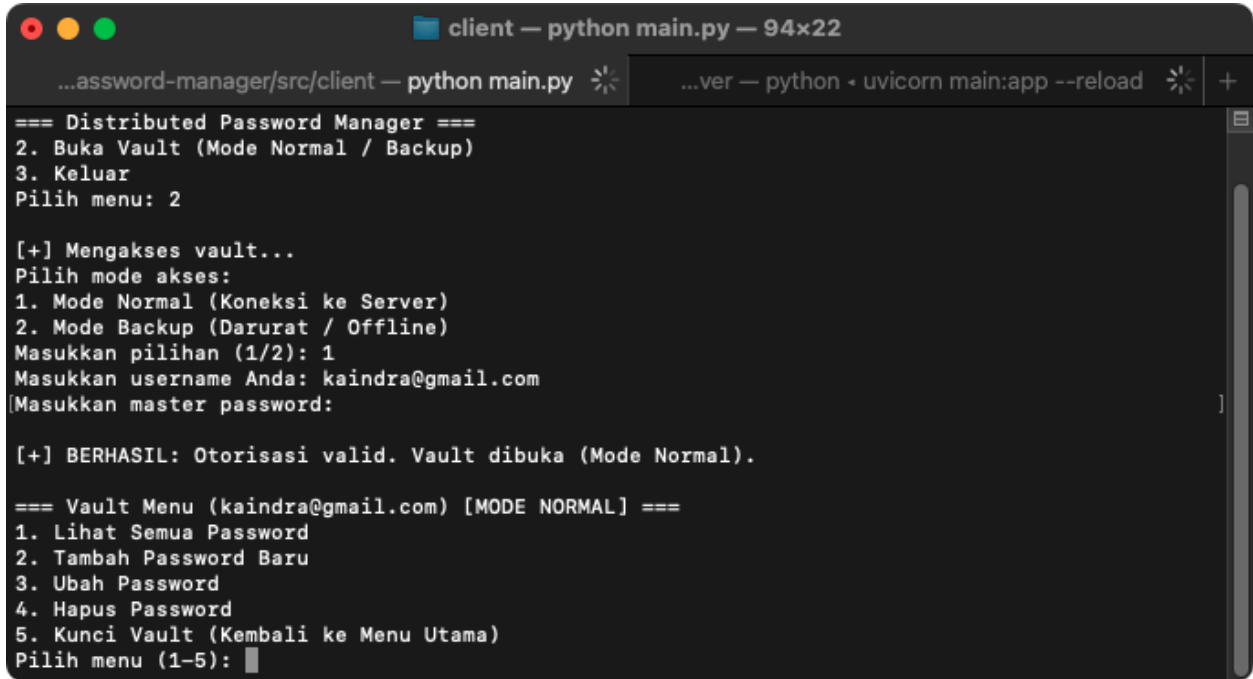
3.3 Pengujian Akses Normal

3.3.1 Rekonstruksi dan Akses *Vault* Valid

Komponen	Keterangan
Tujuan Uji	Memverifikasi penggabungan <i>share</i> yang sah dari klien dan server memulihkan fungsionalitas sistem.
Input	<code>master_password</code> yang benar, serta kondisi server aktif dan dapat diakses.
Prosedur	Buka vault pada Mode Normal menggunakan kredensial yang valid.
Output yang Diharapkan	Sistem berhasil menggunakan kombinasi <i>local share</i> dan <i>server share</i> . <i>Master key</i> berhasil direkonstruksi dari kedua <i>share</i> tersebut, lalu <i>vault</i> berhasil didekripsi dan isinya dapat ditampilkan di antarmuka klien.

a. Hasil

Tampilan Terminal Client



```
client — python main.py — 94x22
...assword-manager/src/client — python main.py
...ver — python • uvicorn main:app --reload

=== Distributed Password Manager ===
2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: 2

[+] Mengakses vault...
Pilih mode akses:
1. Mode Normal (Koneksi ke Server)
2. Mode Backup (Darurat / Offline)
Masukkan pilihan (1/2): 1
Masukkan username Anda: kaina@gmail.com
[Masukkan master password:

[+] BERHASIL: Otorisasi valid. Vault dibuka (Mode Normal).

=== Vault Menu (kaina@gmail.com) [MODE NORMAL] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5):
```

Gambar 3.3.1 *Master key* berhasil direkonstruksi dengan *share* yang valid

b. Analisis

Rekonstruksi kunci dan dekripsi *vault* pada mode normal tervalidasi. Setelah klien berhasil mendekripsi *local share* di memori, `open_vault_normal` memanggil request HTTP GET untuk mendapatkan `server_share`, `vault_ciphertext`, dan `vault_nonce` dari server. *String local share* dan *server share* dieksekusi oleh fungsi `sss.reconstruct_key()` yang mengembalikan susunan biner *master key*. *Master key* ini diteruskan ke `aes_gcm.decrypt_data()` yang memverifikasi tag autentikasi GCM dan memetakan *ciphertext* menjadi *plaintext* bytes, sehingga `json.loads()` dapat mengurainya menjadi dictionary.

3.3.2 Kegagalan Dekripsi Akibat *Share Invalid*

Komponen	Keterangan
Tujuan Uji	Memastikan modifikasi <i>share</i> yang tidak sah merusak integritas master key dan mencegah dekripsi <i>vault</i> .
Input	<code>master_password</code> yang salah atau modifikasi nilai <i>server share</i> di <i>database</i>

Komponen	Keterangan
	<i>server</i> secara paksa.
Prosedur	Masukkan sandi yang salah pada mode normal, atau gunakan basis data server yang nilai <i>share</i> -nya sudah direkayasa.
Output yang Diharapkan	Sistem gagal memvalidasi algoritma enkripsi (dekripsi <i>vault</i> gagal), dan data password pengguna tidak ditampilkan sama sekali di layar terminal.

a. Hasil

Tampilan Terminal Client

```

client — python main.py — 94x22
...assword-manager/src/client — python main.py
...ver — python - uvicorn main:app --reload ... +

[(venv) (base) mac@Kaindras-MacBook-Pro client % python main.py]

=== Distributed Password Manager ===
2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: 2

[+] Mengakses vault...
Pilih mode akses:
1. Mode Normal (Koneksi ke Server)
2. Mode Backup (Darurat / Offline)
Masukkan pilihan (1/2): 1
Masukkan username Anda: kaindra@gmail.com
[Masukkan master password:
[-] Failed to decrypt local share. Incorrect master password?
    Saran: Jika server sedang mati, silakan gunakan Mode Backup.

=== Distributed Password Manager ===
2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: █

```

Gambar 3.3.2 Dekripsi gagal akibat salah email atau *username*

b. Analisis

Penolakan akses akibat *share* yang *invalid* terverifikasi. Jika *server_share* dimanipulasi, *sss.reconstruct_key()* akan menghasilkan susunan biner *master key* yang cacat. Ketika kunci ini diteruskan ke *aes_gcm.decrypt_data()*, AES-128-GCM gagal memvalidasi tag autentikasi. Hal ini memicu *try-except* untuk melempar *exception*, sehingga data password tidak ditampilkan.

3.4 Pengujian Penambahan Data Password

3.4.1 Penambahan dengan Input Manual

Komponen	Keterangan
Tujuan Uji	Memverifikasi bahwa data password yang dimasukkan secara manual berhasil tersimpan ke dalam vault dan tersinkronisasi ke server serta backup lokal.
Input	Nama Layanan: GitHub, Username: peter@mail.com, Metode: a (input manual), Password: PwManualGitHub#1, Catatan: akun kuliah.
Prosedur	Buka vault peter mode normal, pilih menu 2 (Tambah Password Baru); Isi nama layanan, username, pilih metode a, masukkan password dan catatan; Pilih menu 1 (Lihat Semua Password).
Output yang Diharapkan	Muncul pesan [+] Berhasil menambahkan akun untuk GitHub dan vault telah disinkronisasi., akun GitHub tampil di daftar

a. Hasil

Tampilan Terminal Client

```
Windows PowerShell

=== Vault Menu (peter) [MODE NORMAL] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5): 2

[+] Tambah Data Password Baru
Nama Layanan (misal: GitHub): Github
Username / Email: peter@mail.com
Pilih metode password:
a. Input manual
b. Generate otomatis (CSPRNG)
Pilihan (a/b): a
Masukkan password:
Catatan (opsional): akun kuliah
[+] Berhasil menambahkan akun untuk Github dan vault telah disinkronisasi.

=== Vault Menu (peter) [MODE NORMAL] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5): 1

[+] Menampilkan data password yang tersimpan:
-----
Layanan : Github
Username: peter@mail.com
Password: PwManualGitHub#1
Catatan : akun kuliah
```

Gambar 3.4.1 Tampilan Terminal Client

b. Analisis

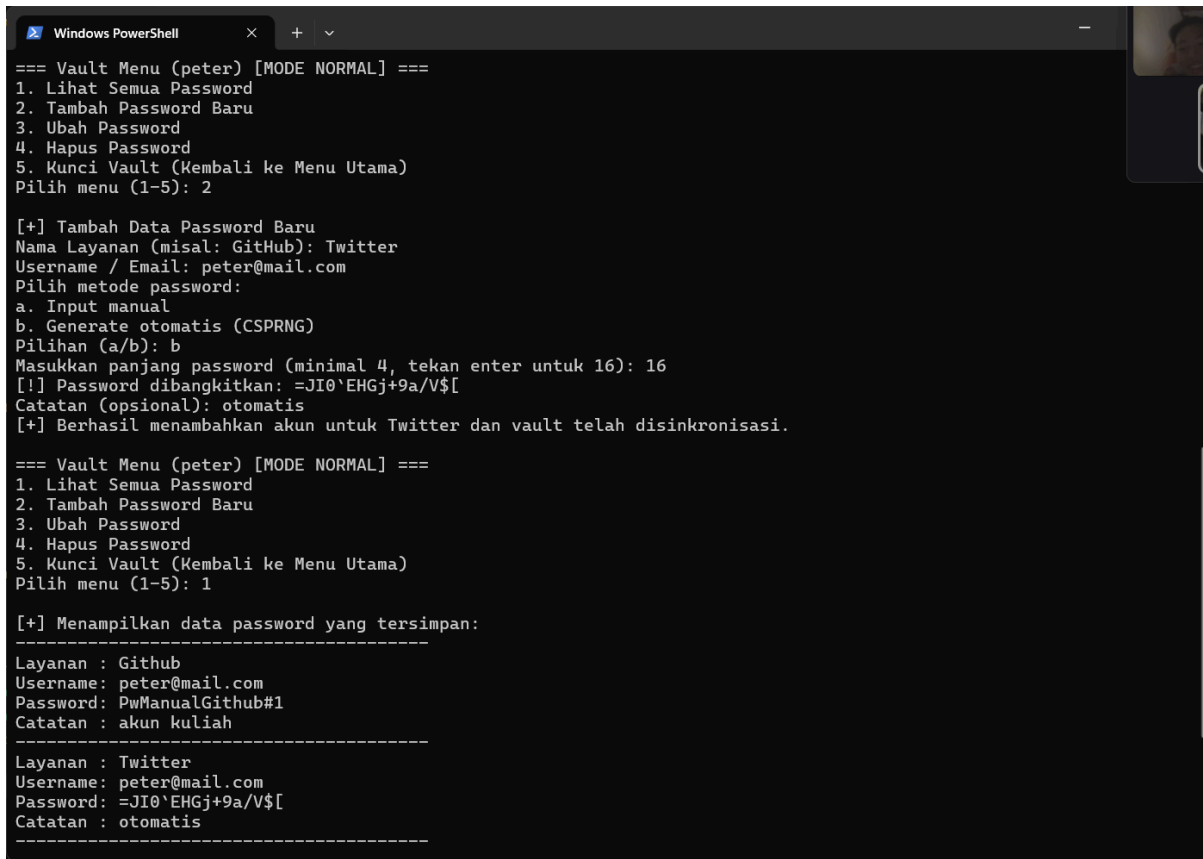
Pada metode input manual, password diambil langsung dari masukan pengguna melalui `getpass.getpass()` pada `client/main.py` sehingga tidak tampil di layar saat diketik. Nilai password tersebut, bersama username dan catatan, disusun menjadi satu entri `vault_data[service] = {"username": user_id, "password": pwd, "catatan": catatan}` dengan nama layanan sebagai kunci. Dengan demikian data password yang dimasukkan secara manual berhasil terbentuk sebagai entri baru di dalam struktur vault dan langsung tampil saat daftar password dibuka.

3.4.2 Penambahan dengan Pembangkitan Otomatis

Komponen	Keterangan
Tujuan Uji	Memverifikasi bahwa password dapat dibangkitkan otomatis sesuai panjang yang diminta dan memenuhi syarat kompleksitas.
Input	Nama Layanan: Twitter, Username: peter@mail.com, Metode: b (generate otomatis), Panjang: 16, Catatan: dibangkitkan otomatis.
Prosedur	1. Pilih menu 2 (Tambah Password Baru). 2. Isi nama layanan dan username. 3. Pilih metode b, masukkan panjang 16. 4. Pilih menu 1 (Lihat Semua Password).
Output yang Diharapkan	Baris [!] Password dibangkitkan: menampilkan password sepanjang 16 karakter yang memuat huruf besar, huruf kecil, angka, dan simbol. Akun Twitter tampil dengan password tersebut.

a. Hasil

Tampilan Terminal Client



```
Windows PowerShell
=== Vault Menu (peter) [MODE NORMAL] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5): 2

[+] Tambah Data Password Baru
Nama Layanan (misal: GitHub): Twitter
Username / Email: peter@mail.com
Pilih metode password:
a. Input manual
b. Generate otomatis (CSPRNG)
Pilihan (a/b): b
Masukkan panjang password (minimal 4, tekan enter untuk 16): 16
[!] Password dibangkitkan: =JI0`EHGj+9a/V$[
Catatan (opsional): otomatis
[+] Berhasil menambahkan akun untuk Twitter dan vault telah disinkronisasi.

=== Vault Menu (peter) [MODE NORMAL] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5): 1

[+] Menampilkan data password yang tersimpan:
-----
Layanan : Github
Username: peter@mail.com
Password: PwManualGithub#1
Catatan : akun kuliah
-----
Layanan : Twitter
Username: peter@mail.com
Password: =JI0`EHGj+9a/V$[
Catatan : otomatis
-----
```

Gambar 3.4.2 Tampilan Terminal Client

b. Analisis

Pada metode generate otomatis, panjang yang diminta pengguna dibaca lalu diteruskan ke `generate_secure_password(max(4, panjang))` pada `client/cli/vault_manager.py`. Fungsi ini membangun password dengan `secrets.choice (CSPRNG)` sebanyak `length` iterasi dari himpunan `string.ascii_letters + string.digits + string.punctuation`, sehingga jumlah karakter keluaran sama persis dengan panjang yang diminta (16 karakter). Pembangkitan diulang hingga password dipastikan memuat minimal satu huruf kecil, huruf besar, angka, dan simbol, lalu disimpan ke struktur vault dengan cara yang sama seperti input manual.

3.4.3 Enkripsi Ulang Vault dan Pembaruan Backup Vault

Komponen	Keterangan
Tujuan Uji	Membuktikan bahwa setiap penambahan data menghasilkan nonce baru, sehingga vault benar-benar dienkrpsi ulang sebelum dikirim ke server, serta bahwa backup vault lokal ikut diperbarui bersamaan dengan server pada setiap perubahan di mode normal.
Input	Tidak ada data baru. Pengamatan memakai operasi penambahan dari iv.1 (GitHub) dan iv.2 (Twitter).
Prosedur	1. Lakukan penambahan 3.4.1, jalankan cek_server, catat Nonce, dan juga backup_vault_cipher. 2. Lakukan penambahan 3.4.2, jalankan cek_server sekali lagi. 3. Bandingkan ketiga nilai Nonce, dan backup_vault_cipher.
Output yang Diharapkan	Nilai Nonce pada server berbeda di setiap pengamatan, membuktikan vault dienkrpsi ulang pada tiap operasi penambahan. Nilai backup_vault_cipher berubah setelah penambahan, dan cek_server menunjukkan ciphertext server juga berubah, membuktikan keduanya tersinkronisasi dalam satu operasi.

a. Hasil

Tampilan Database Server (Sebelum)

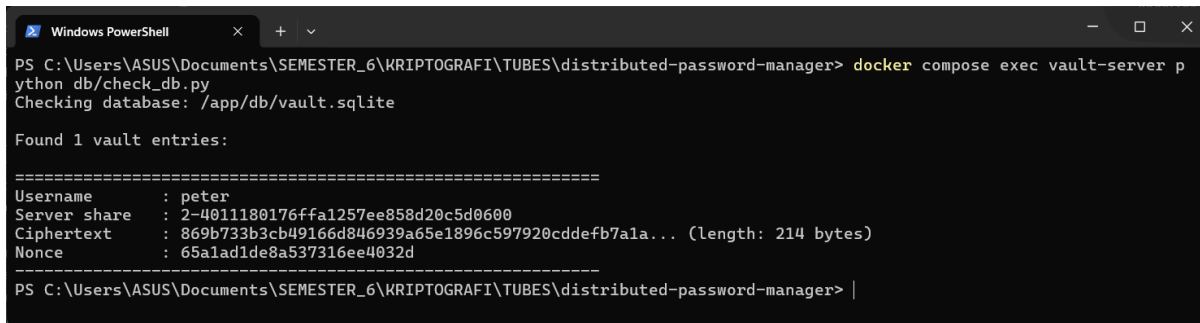
```
Windows PowerShell
PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> docker compose exec vault-server p
ython db/check_db.py
Checking database: /app/db/vault.sqlite

Found 1 vault entries:

=====
Username      : peter
Server share  : 2-4011180176ffa1257ee858d20c5d0600
Ciphertext    : 8d942e6208eb6b1c52d78da5fd7dd4865e50e0a354f16e31... (length: 116 bytes)
Nonce        : 3216af2e525316db53c84e74
=====
PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> |
```

Gambar 3.4.3 Tampilan Database Server pada Uji 3.4.1

Tampilan Database Server (Sesudah)



```
PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> docker compose exec vault-server python db/check_db.py
Checking database: /app/db/vault.sqlite

Found 1 vault entries:
=====
Username      : peter
Server share  : 2-4011180176ffa1257ee858d20c5d0600
Ciphertext    : 869b733b3cb49166d846939a65e1896c597920cddefb7a1a... (length: 214 bytes)
Nonce         : 65a1ad1de8a537316ee4032d
=====
PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> |
```

Gambar 3.4.4 Tampilan Database Server pada Uji 3.4.2

Tampilan Backup Lokal (Sebelum)



```
{
  "kdf_salt": "6b15608eb8e23571b98fbcd94b845fce",
  "local_share_cipher": "2ab3d7f5ab889dc9d1c4c99abe07c41...",
  "local_share_nonce": "716059f6614da794b4141f72",
  "backup_vault_cipher": "8d942e6208eb6b1c52d78da5fd7dd4865e...",
  "backup_vault_nonce": "3216af2e525316db53c84e74"
}
```

Gambar 3.4.5 Tampilan Backup Lokal pada Uji 3.4.1

Tampilan Backup Lokal (Sesudah)



```
{
  "kdf_salt": "6b15608eb8e23571b98fbcd94b845fce",
  "local_share_cipher": "2ab3d7f5ab889dc9d1c4c99abe07c413f5e19c79270c989169c80f23d90f...",
  "local_share_nonce": "716059f6614da794b4141f72",
  "backup_vault_cipher": "869b733b3cb49166d846939a65e1896c597920cddefb7a1a2428978b5ad8...",
  "backup_vault_nonce": "65a1ad1de8a537316ee4032d"
}
```

Gambar 3.4.6 Tampilan Backup Lokal pada Uji 3.4.2

b. Analisis

Setiap penambahan memicu `update_vault()` pada `client/cli/vault_manager.py`, yang menyusun ulang seluruh isi vault menjadi JSON lalu memanggil `aes_gcm.encrypt_data()`. Fungsi enkripsi tersebut membuat nonce 12-byte baru pada setiap pemanggilan melalui `secrets.token_bytes(NONCE_LENGTH)`, sehingga Nonce dan Ciphertext yang dikirim ke server melalui `PUT /api/vault/{username}` selalu berbeda. Ini menegaskan vault tidak disimpan secara

statik, melainkan dienkripsi ulang utuh sebelum dikirim, sekaligus memenuhi syarat AES-GCM agar nonce tidak pernah dipakai ulang pada kunci yang sama. Pada operasi yang sama, ciphertext dan nonce tersebut juga disimpan ke backup lokal melalui `storage.save_backup_vault()` sebelum permintaan PUT dikirim, sehingga `client_data.json` dan server selalu memuat versi vault yang identik. Perbandingan baseline, setelah 3.4.1, dan setelah 3.4.2 memperlihatkan keduanya berubah bersamaan di setiap penambahan.

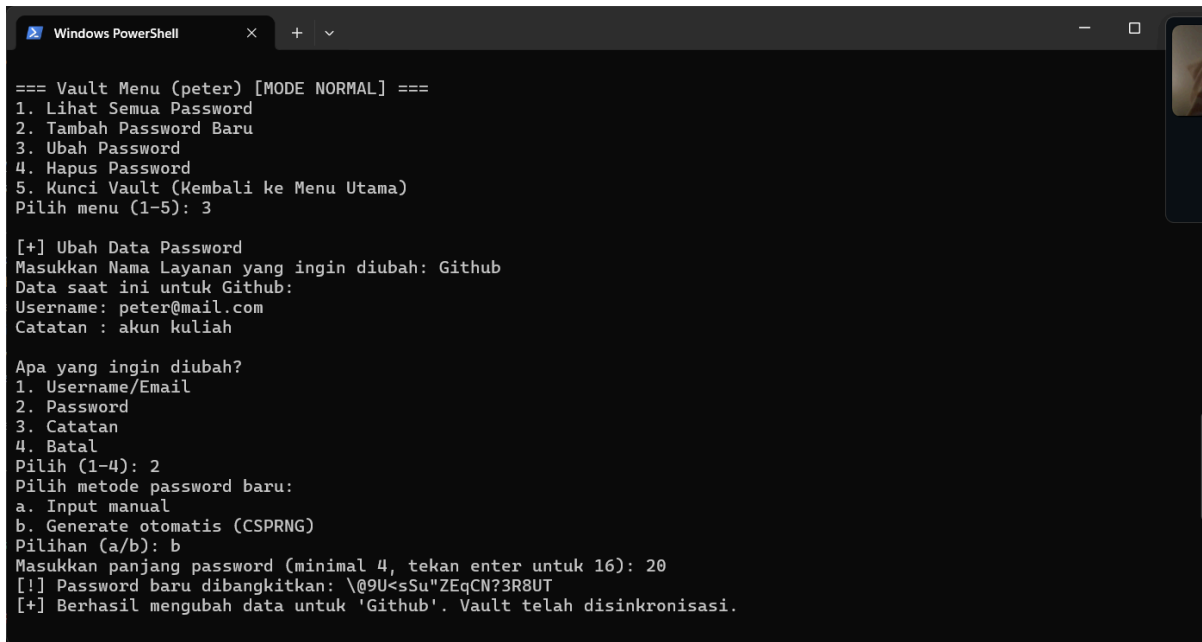
3.5 Pengujian Pengubahan dan Penghapusan Data Password

3.5.1 Pengubahan Data Password

Komponen	Keterangan
Tujuan Uji	Memverifikasi bahwa pengubahan data password berhasil tersimpan setelah vault dienkripsi ulang.
Input	Layanan yang diubah: GitHub, Bidang: 2 (Password), Metode: b (generate otomatis), Panjang: 20.
Prosedur	1. Jalankan <code>cek_server</code> , catat Nonce. 2. Pilih menu 3 (Ubah Password). 3. Masukkan nama layanan GitHub, pilih 2, pilih metode b, panjang 20. 4. Pilih menu 1 (Lihat) untuk memastikan password berganti.
Output yang Diharapkan	Muncul pesan [+] Berhasil mengubah data untuk 'GitHub'. Vault telah disinkronisasi., password GitHub berganti, nilai Ciphertext dan Nonce server berubah, serta backup lokal berubah.

a. Hasil

Tampilan Terminal Client

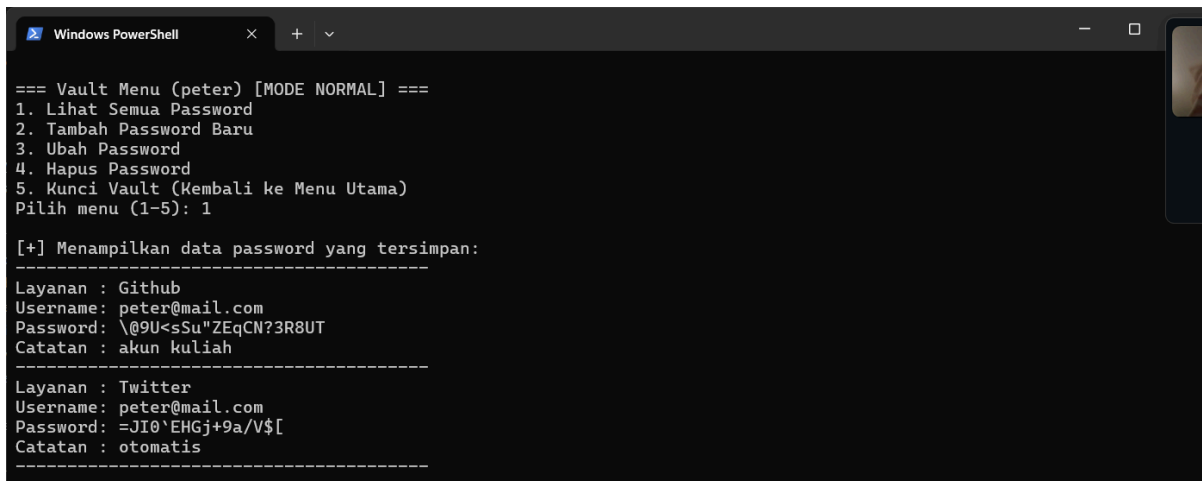


```
Windows PowerShell
=== Vault Menu (peter) [MODE NORMAL] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5): 3

[+] Ubah Data Password
Masukkan Nama Layanan yang ingin diubah: Github
Data saat ini untuk Github:
Username: peter@mail.com
Catatan : akun kuliah

Apa yang ingin diubah?
1. Username/Email
2. Password
3. Catatan
4. Batal
Pilih (1-4): 2
Pilih metode password baru:
a. Input manual
b. Generate otomatis (CSPRNG)
Pilihan (a/b): b
Masukkan panjang password (minimal 4, tekan enter untuk 16): 20
[!] Password baru dibangkitkan: \@9U<sSu"ZEqCN?3R8UT
[+] Berhasil mengubah data untuk 'Github'. Vault telah disinkronisasi.
```

Gambar 3.5.1 Tampilan Terminal Client Ketika Mengubah Password

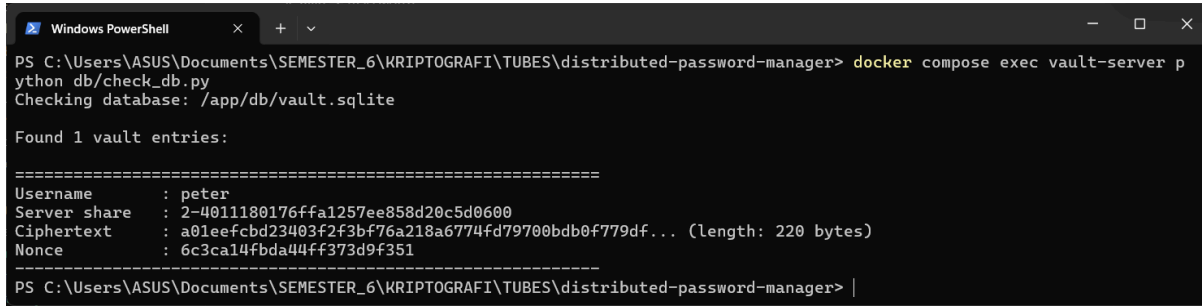


```
Windows PowerShell
=== Vault Menu (peter) [MODE NORMAL] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5): 1

[+] Menampilkan data password yang tersimpan:
-----
Layanan : Github
Username: peter@mail.com
Password: \@9U<sSu"ZEqCN?3R8UT
Catatan : akun kuliah
-----
Layanan : Twitter
Username: peter@mail.com
Password: =JI0`EHGj+9a/V$[
Catatan : otomatis
-----
```

Gambar 3.5.2 Tampilan Terminal Client Menampilkan Data Password

Tampilan Database Server



```
PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> docker compose exec vault-server python db/check_db.py
Checking database: /app/db/vault.sqlite

Found 1 vault entries:
=====
Username      : peter
Server share   : 2-4011180176ffa1257ee858d20c5d0600
Ciphertext    : a01eefcbd23403f2f3bf76a218a6774fd79700bdb0f779df... (length: 220 bytes)
Nonce         : 6c3ca14fbda44ff373d9f351
=====
PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> |
```

Gambar 3.5.3 Tampilan Database Server setelah Mengubah Password

b. Analisis

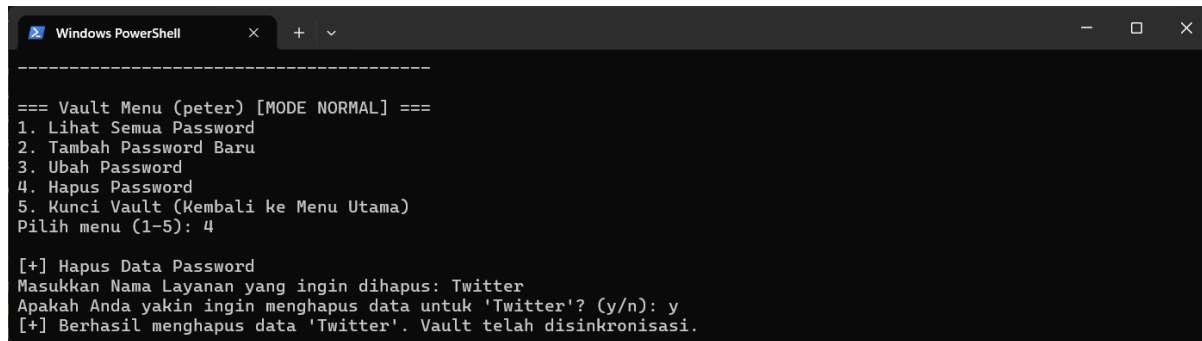
Operasi ubah memodifikasi entri pada struktur vault di memori, lalu menjalankan jalur yang sama dengan penambahan, yaitu `update_vault()` yang memanggil `aes_gcm.encrypt_data()` dan `storage.save_backup_vault()`. Karena enkripsi mencakup seluruh vault dengan nonce baru, perubahan satu bidang password tetap menghasilkan ciphertext dan nonce baru di server maupun backup lokal.

3.5.2 Penghapusan Data Password

Komponen	Keterangan
Tujuan Uji	Memverifikasi bahwa data yang dihapus tidak lagi muncul saat vault dibuka kembali.
Input	Layanan yang dihapus: Twitter, Konfirmasi: y.
Prosedur	1. Pilih menu 4 (Hapus Password), masukkan Twitter, konfirmasi y. 2. Pilih menu 5 (Kunci Vault). 3. Buka kembali vault peter mode normal. 4. Pilih menu 1 (Lihat Semua Password).
Output yang Diharapkan	Muncul pesan [+] Berhasil menghapus data 'Twitter'. Vault telah disinkronisasi., dan setelah vault dibuka kembali akun Twitter tidak muncul. Nilai Ciphertext/Nonce server dan backup lokal berubah.

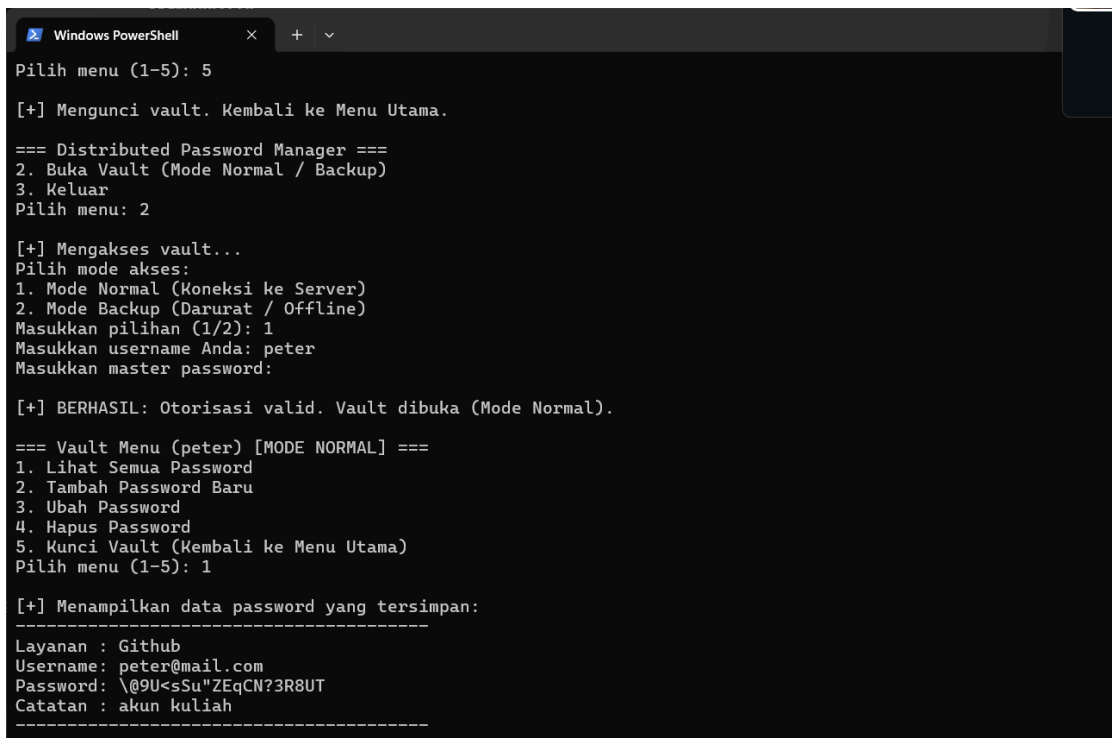
a. Hasil

Tampilan Terminal Client



```
-----  
=== Vault Menu (peter) [MODE NORMAL] ===  
1. Lihat Semua Password  
2. Tambah Password Baru  
3. Ubah Password  
4. Hapus Password  
5. Kunci Vault (Kembali ke Menu Utama)  
Pilih menu (1-5): 4  
  
[+] Hapus Data Password  
Masukkan Nama Layanan yang ingin dihapus: Twitter  
Apakah Anda yakin ingin menghapus data untuk 'Twitter'? (y/n): y  
[+] Berhasil menghapus data 'Twitter'. Vault telah disinkronisasi.
```

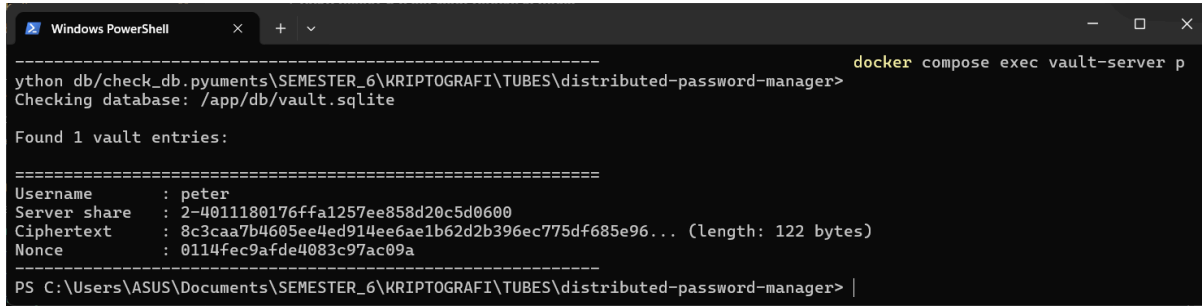
Gambar 3.5.4 Tampilan Terminal Client Ketika Menghapus Password



```
Pilih menu (1-5): 5  
  
[+] Mengunci vault. Kembali ke Menu Utama.  
  
=== Distributed Password Manager ===  
2. Buka Vault (Mode Normal / Backup)  
3. Keluar  
Pilih menu: 2  
  
[+] Mengakses vault...  
Pilih mode akses:  
1. Mode Normal (Koneksi ke Server)  
2. Mode Backup (Darurat / Offline)  
Masukkan pilihan (1/2): 1  
Masukkan username Anda: peter  
Masukkan master password:  
  
[+] BERHASIL: Otorisasi valid. Vault dibuka (Mode Normal).  
  
=== Vault Menu (peter) [MODE NORMAL] ===  
1. Lihat Semua Password  
2. Tambah Password Baru  
3. Ubah Password  
4. Hapus Password  
5. Kunci Vault (Kembali ke Menu Utama)  
Pilih menu (1-5): 1  
  
[+] Menampilkan data password yang tersimpan:  
-----  
Layanan : Github  
Username: peter@mail.com  
Password: \@9U<sSu"ZEqCN?3R8UT  
Catatan : akun kuliah  
-----
```

Gambar 3.5.5 Tampilan Terminal Client Menampilkan Data Password

Tampilan Database Server



```
-----
docker compose exec vault-server p
ython db/check_db.pyuments\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager>
Checking database: /app/db/vault.sqlite

Found 1 vault entries:
=====
Username      : peter
Server share  : 2-4011180176ffa1257ee858d20c5d0600
Ciphertext    : 8c3caa7b4605ee4ed914ee6ae1b62d2b396ec775df685e96... (length: 122 bytes)
Nonce         : 0114fec9afde4083c97ac09a
=====
PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> |
```

Gambar 3.5.6 Tampilan Database Server setelah Menghapus Password

b. Analisis

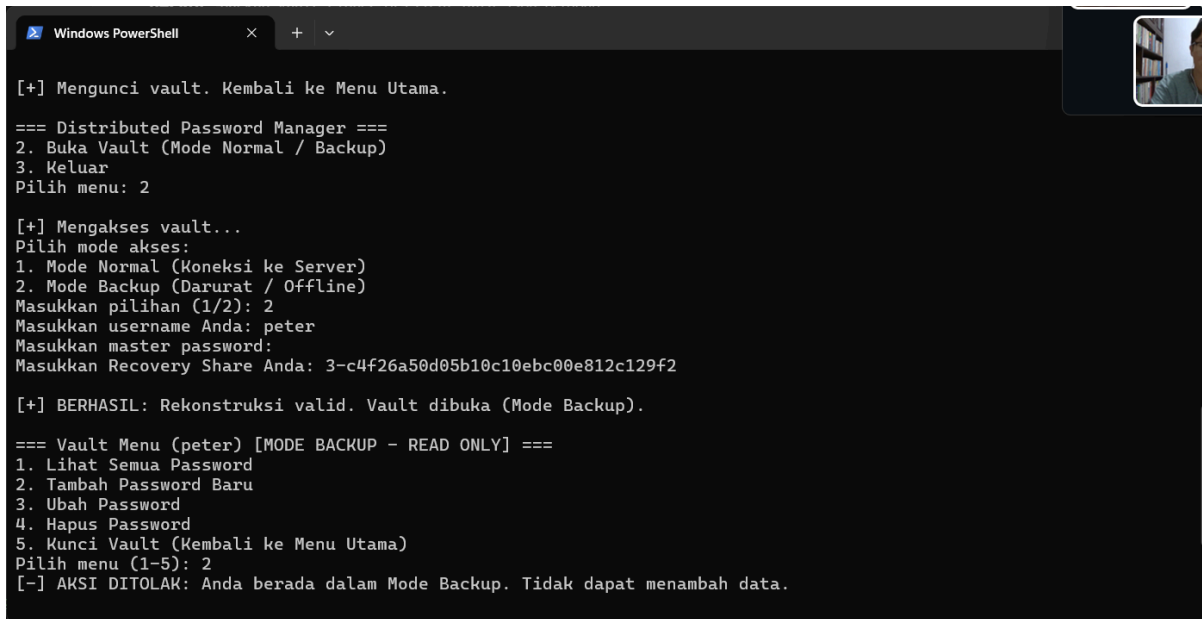
Penghapusan menghilangkan entri dari struktur vault, kemudian `update_vault()` mengenkripsi ulang vault tanpa entri tersebut dan menyimpannya ke server serta backup lokal. Pembuktian utama adalah membuka kembali vault dari server: karena Twitter tidak lagi muncul, penghapusan benar-benar terjadi pada data tersimpan, bukan sekadar tampilan sementara.

3.5.3 Validasi Perubahan Hanya pada Mode Normal

Komponen	Keterangan
Tujuan Uji	Membuktikan bahwa mode backup bersifat read-only dan tidak menulis ke server maupun backup lokal.
Input	Master password: RahasiaKuat123!, Recovery Share hasil inisialisasi, percobaan menu 2, 3, dan 4.
Prosedur	1. Pilih menu 2 (Buka Vault), pilih mode 2 (Backup). 2. Masukkan master password dan Recovery Share. 3. Coba menu 2 (Tambah), 3 (Ubah), dan 4 (Hapus). 4. Jalankan <code>cek_server</code> dan periksa <code>client_data.json</code> sebelum dan sesudah percobaan.
Output yang Diharapkan	Header menampilkan [MODE BACKUP - READ ONLY], setiap percobaan tulis ditolak dengan pesan [-] AKSI DITOLAK: Anda berada dalam Mode Backup, serta nilai Ciphertext/Nonce server dan <code>client_data.json</code> tidak berubah.

a. Hasil

Tampilan Terminal Client



```
Windows PowerShell

[+] Mengunci vault. Kembali ke Menu Utama.

=== Distributed Password Manager ===
2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: 2

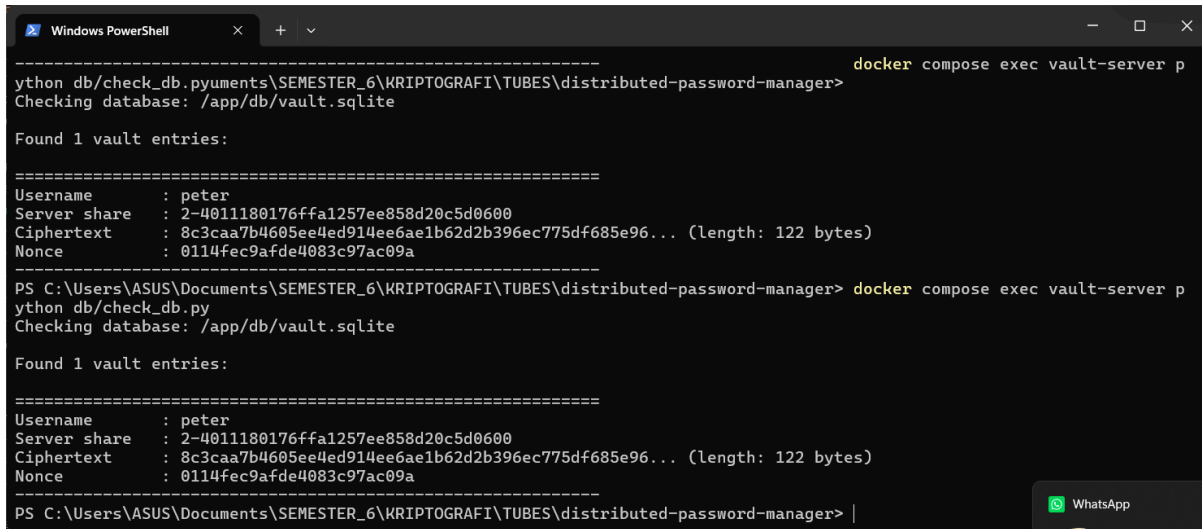
[+] Mengakses vault...
Pilih mode akses:
1. Mode Normal (Koneksi ke Server)
2. Mode Backup (Darurat / Offline)
Masukkan pilihan (1/2): 2
Masukkan username Anda: peter
Masukkan master password:
Masukkan Recovery Share Anda: 3-c4f26a50d05b10c10ebc00e812c129f2

[+] BERHASIL: Rekonstruksi valid. Vault dibuka (Mode Backup).

=== Vault Menu (peter) [MODE BACKUP - READ ONLY] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5): 2
[-] AKSI DITOLAK: Anda berada dalam Mode Backup. Tidak dapat menambah data.
```

Gambar 3.5.7 Tampilan Terminal Client dalam Mode Backup

Tampilan Database Server



```
Windows PowerShell

----- docker compose exec vault-server p
ython db/check_db.py
Checking database: /app/db/vault.sqlite

Found 1 vault entries:

=====
Username      : peter
Server share  : 2-4011180176ffa1257ee858d20c5d0600
Ciphertext    : 8c3caa7b4605ee4ed914ee6ae1b62d2b396ec775df685e96... (length: 122 bytes)
Nonce         : 0114fec9afde4083c97ac09a
-----

PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> docker compose exec vault-server p
ython db/check_db.py
Checking database: /app/db/vault.sqlite

Found 1 vault entries:

=====
Username      : peter
Server share  : 2-4011180176ffa1257ee858d20c5d0600
Ciphertext    : 8c3caa7b4605ee4ed914ee6ae1b62d2b396ec775df685e96... (length: 122 bytes)
Nonce         : 0114fec9afde4083c97ac09a
-----

PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> |
```

Gambar 3.5.8 Tampilan Database Server dalam Mode Backup

b. Analisis

Mode *backup* dibuka melalui `open_vault_backup()` yang merekonstruksi master key dari local share dan recovery share hanya untuk dekripsi backup lokal. Pada mode ini menu tulis diblokir sehingga `update_vault()` tidak pernah dipanggil, dan tidak ada permintaan PUT ke server

maupun penulisan ke client_data.json. Hasilnya, kondisi server dan backup lokal tetap identik sebelum dan sesudah percobaan, menegaskan pembaruan hanya terjadi di mode normal.

3.6 Pengujian Penyimpanan Vault di Server

3.6.1 Vault Tersimpan sebagai BLOB

Komponen	Keterangan
Tujuan Uji	Membuktikan bahwa vault terenkripsi disimpan pada SQLite sebagai BLOB biner, bukan teks terbaca.
Input	Tidak ada input client. Pemeriksaan dilakukan langsung pada basis data server.
Prosedur	1. Jalankan cek_server dan amati baris Ciphertext. 2. Tampilkan skema tabel melalui kueri sqlite_master untuk kolom vault.
Output yang Diharapkan	Baris Ciphertext : ... (length: NN bytes) menampilkan data biner, dan skema memuat vault_ciphertext BLOB NOT NULL serta vault_nonce BLOB NOT NULL.

a. Hasil

Tampilan Database Server

```
Windows PowerShell
-----
docker compose exec vault-server p
python db/check_db.py puments\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager>
Checking database: /app/db/vault.sqlite

Found 1 vault entries:

=====
Username      : peter
Server share  : 2-4011180176ffa1257ee858d20c5d0600
Ciphertext    : 8c3caa7b4605ee4ed914ee6ae1b62d2b396ec775df685e96... (length: 122 bytes)
Nonce         : 0114fec9afde4083c97ac09a
=====
```

Gambar 3.6.1 Tampilan Database Server dalam Mode Backup

Tampilan Skema Tabel

```
Windows PowerShell
PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> python -c "import sqlite3; c=sqlite3.connect('src/server/db/vault.sqlite'); print(c.execute('SELECT sql FROM sqlite_master WHERE name=?', ('vault',)).fetchone()[0])"
CREATE TABLE vault (
  username TEXT PRIMARY KEY,
  server_share TEXT NOT NULL,
  vault_ciphertext BLOB NOT NULL,
  vault_nonce BLOB NOT NULL
)
PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> |
```

Gambar 3.6.2 Definisi Kolom bertipe BLOB pada Tabel Vault

b. Analisis

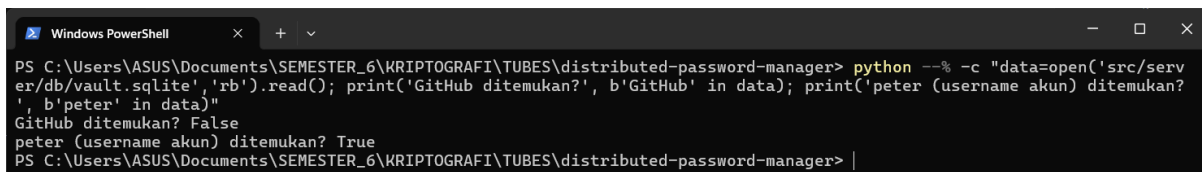
Server hanya menerima *ciphertext* dan nonce dalam bentuk hex dari client, lalu menyimpannya sebagai kolom bertipe BLOB. Karena yang tersimpan adalah keluaran AES-128-GCM, isinya berupa data biner acak yang tidak dapat dibaca tanpa master key. Server tidak memiliki akses ke master key, sehingga tidak dapat mendekripsi BLOB tersebut.

3.6.2 Validasi Penyimpanan Server

Komponen	Keterangan
Tujuan Uji	Membuktikan bahwa server tidak menyimpan nama layanan, username akun, password, atau catatan sebagai baris plaintext terpisah.
Input	Tidak ada input client. Pemeriksaan dilakukan pada file basis data mentah dan skema tabel.
Prosedur	1. Cari string GitHub pada file vault.sqlite mentah. 2. Cari string peter pada file yang sama. 3. Tampilkan skema tabel vault.
Output yang Diharapkan	GitHub ditemukan? False (isi vault terenkripsi), peter ditemukan? True (username akun adalah primary key), dan tabel vault hanya memiliki 4 kolom: username, server_share, vault_ciphertext, vault_nonce.

a. Hasil

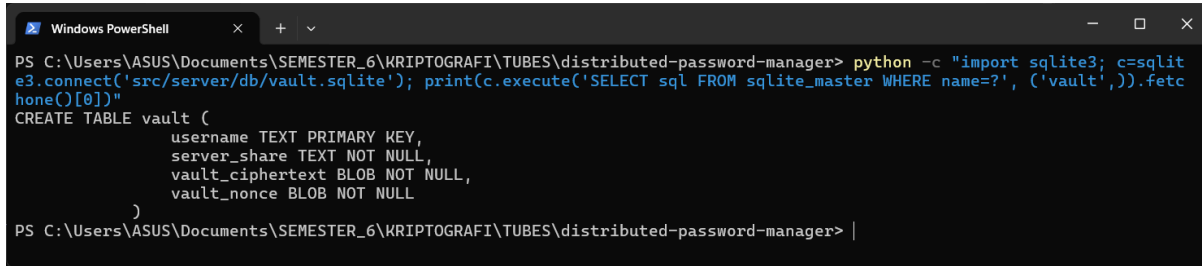
Tampilan Pencarian String



```
PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> python --% -c "data=open('src/server/db/vault.sqlite','rb').read(); print('GitHub ditemukan?', b'GitHub' in data); print('peter (username akun) ditemukan?', b'peter' in data)"
GitHub ditemukan? False
peter (username akun) ditemukan? True
PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> |
```

Gambar 3.6.3 Hasil Pencarian Github dan peter

Tampilan Skema Tabel



```
PS C:\Users\ASUS\Documents\SEMESTER_6\KRIPTOGRAFI\TUBES\distributed-password-manager> python -c "import sqlite3; c=sqlite3.connect('src/server/db/vault.sqlite'); print(c.execute('SELECT sql FROM sqlite_master WHERE name=?', ('vault',)).fetchone()[0])"
CREATE TABLE vault (
  username TEXT PRIMARY KEY,
  server_share TEXT NOT NULL,
  vault_ciphertext BLOB NOT NULL,
  vault_nonce BLOB NOT NULL
)
```

Gambar 3.6.4 Semua Kolom pada Tabel Vault

b. Analisis

Tabel vault tidak menyediakan kolom untuk nama layanan, password, maupun catatan. Seluruh isi vault dikemas menjadi satu BLOB terenkripsi, sehingga string seperti GitHub tidak pernah muncul dalam bentuk plaintext pada file basis data. Hanya username akun yang tersimpan sebagai teks karena berfungsi sebagai primary key untuk identifikasi entri, dan ini bukan bagian dari isi rahasia vault.

3.6.3 Validasi Penerimaan Server

Komponen	Keterangan
Tujuan Uji	Membuktikan bahwa server tidak pernah menerima master key, local share, recovery share, maupun isi vault dalam bentuk plaintext.
Input	Pengamatan payload <code>init_vault()</code> dan <code>update_vault()</code> pada <code>src/client/cli/vault_manager.py</code> , serta log server saat inisialisasi atau penambahan.
Prosedur	1. Periksa isi payload pada <code>init_vault()</code> dan <code>update_vault()</code>. 2. Amati log server saat operasi tulis dilakukan. 3. Jalankan <code>cek_server</code> dan amati kolom Server share.
Output yang Diharapkan	Payload hanya berisi <code>server_share</code> (format 2-...) beserta ciphertext dan nonce hex. Log server tidak memuat master password, master key utuh, local share, maupun recovery share. Kolom Server share menampilkan satu share, bukan master key utuh.

a. Hasil

Tampilan Init Vault

```
Windows PowerShell
=== Distributed Password Manager ===
1. Buat Vault Baru (Inisialisasi)
3. Keluar
Pilih menu: 1

[+] Memulai pembuatan vault baru...
Masukkan username Anda: peter
Masukkan master password:
Konfirmasi master password:

[!] VAULT BERHASIL DIBUAT!
=====
RECOVERY SHARE ANDA:
3-b9f3732da3208c1c7ede67681f5b1450
=====
[!] PERINGATAN KELANGSUNGAN HIDUP VAULT ANDA:
Simpan recovery share ini di tempat yang aman (copy-paste).
Jika server mati dan Anda kehilangan share ini, vault tidak bisa dibuka!

[+] Memproses Kriptografi Visual untuk Recovery Share...
[!] Berhasil! 3 file gambar telah dibuat di direktori saat ini:
- recovery_peter_share1.png (Simpan di tempat A)
- recovery_peter_share2.png (Simpan di tempat B)
- recovery_peter_combined.png (Simulasi penggabungan, coba scan QR-nya!)
=====

[+] Building 1.7s (12/12) FINISHED
vault-server-1 | Starting server and preparing database
vault-server-1 | INFO:      Application startup complete.
vault-server-1 | INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
vault-server-1 |
vault-server-1 | [SERVER LOG] INCOMING INITIALIZATION PAYLOAD
vault-server-1 |   Username   : peter
vault-server-1 |   Server Share: 2-69a870dd147ffefc8dc02ad730944019...
vault-server-1 |   Ciphertext  : 5325a864953961ca21446eafe29010325920... (length: 36 hex chars)
vault-server-1 |   Nonce       : 4181005d98ade0c72c4d6601
vault-server-1 | -----
vault-server-1 |
vault-server-1 | INFO:      172.19.0.1:35030 - "POST /api/vault/init HTTP/1.1" 200 OK
```

Gambar 3.6.5 Tampilan Klien dan Server pada saat Init Vault

Tampilan Payload Server

```
vault-server-1 | Starting server and preparing database
vault-server-1 | INFO:      Application startup complete.
vault-server-1 | INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
vault-server-1 | INFO:      172.19.0.1:52040 - "GET /api/vault/peter HTTP/1.1" 200 OK
vault-server-1 |
vault-server-1 | [SERVER LOG] INCOMING UPDATE PAYLOAD
vault-server-1 |   Username   : peter
vault-server-1 |   Ciphertext: 271b9638de20179243a4d6bab935295223941e9... (length: 620 hex chars)
vault-server-1 |   Nonce      : bd67f037d9d831ed7f8e4853
vault-server-1 | -----
vault-server-1 |
vault-server-1 | INFO:      172.19.0.1:51540 - "PUT /api/vault/peter HTTP/1.1" 200 OK
```

Gambar 3.6.6 Log Payload Masuk Tanpa Data Sensitif Plaintext

b. Analisis

Master key dibangkitkan di sisi client lalu dipecah menjadi tiga share melalui `sss.split_key()` yang memakai skema Shamir Secret Sharing dengan threshold (2, 3) pada array 16-byte. Dari ketiga share, hanya `server_share` yang dikirim ke server, sedangkan `local share` dan `recovery share` tetap di sisi client. Payload `init_vault()` berisi `username`, `server_share`, serta `ciphertext` dan `nonce vault`, sementara `update_vault()` hanya mengirim `ciphertext` dan `nonce`. Master key tidak pernah dikirim, ia hanya direkonstruksi sementara di memori client melalui `sss.reconstruct_key()` saat membuka vault, lalu dilepas setelah operasi selesai. Dengan begitu, satu share di server tidak cukup untuk merekonstruksi master key, sesuai prinsip zero-knowledge.

3.7 Pengujian Mode Backup

3.7.1 Akses *Vault* secara Luring

Komponen	Keterangan
Tujuan Uji	Memverifikasi sistem dapat memulihkan akses luring dengan <i>share</i> milik pengguna.
Input	<code>master_password</code> dan <code>recovery_share</code> yang benar.
Prosedur	Simulasikan kondisi server tidak dapat diakses dengan tidak run server. Buka <i>vault</i> dan pilih opsi mode <i>backup</i> . Kemudian masukkan <i>master password</i> dan

Komponen	Keterangan
	<i>recovery share</i> .
Output yang Diharapkan	Sistem berhasil merekonstruksi <i>master key</i> menggunakan kombinasi <i>local share</i> dan <i>recovery share</i> . Sistem menggunakan <i>backup vault</i> lokal terenkripsi sebagai sumber utama <i>vault</i> , dan data <i>password</i> berhasil didekripsi serta ditampilkan di layar.

a. Hasil

Tampilan Terminal Client

```

client — python main.py — 94x22
...assword-manager/src/client — python main.py
...ted-password-manager/src/server — -zsh

2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: 2

[+] Mengakses vault...
Pilih mode akses:
1. Mode Normal (Koneksi ke Server)
2. Mode Backup (Darurat / Offline)
Masukkan pilihan (1/2): 2
Masukkan username Anda: kaindra@gmail.com
Masukkan master password:
Masukkan Recovery Share Anda: 3-fae35edc47c1312bd14911b41f6338b7

[+] BERHASIL: Rekonstruksi valid. Vault dibuka (Mode Backup).

=== Vault Menu (kaindra@gmail.com) [MODE BACKUP - READ ONLY] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5):

```

Gambar 3.7.1 Vault berhasil dibuka dengan mode *backup*

b. Analisis

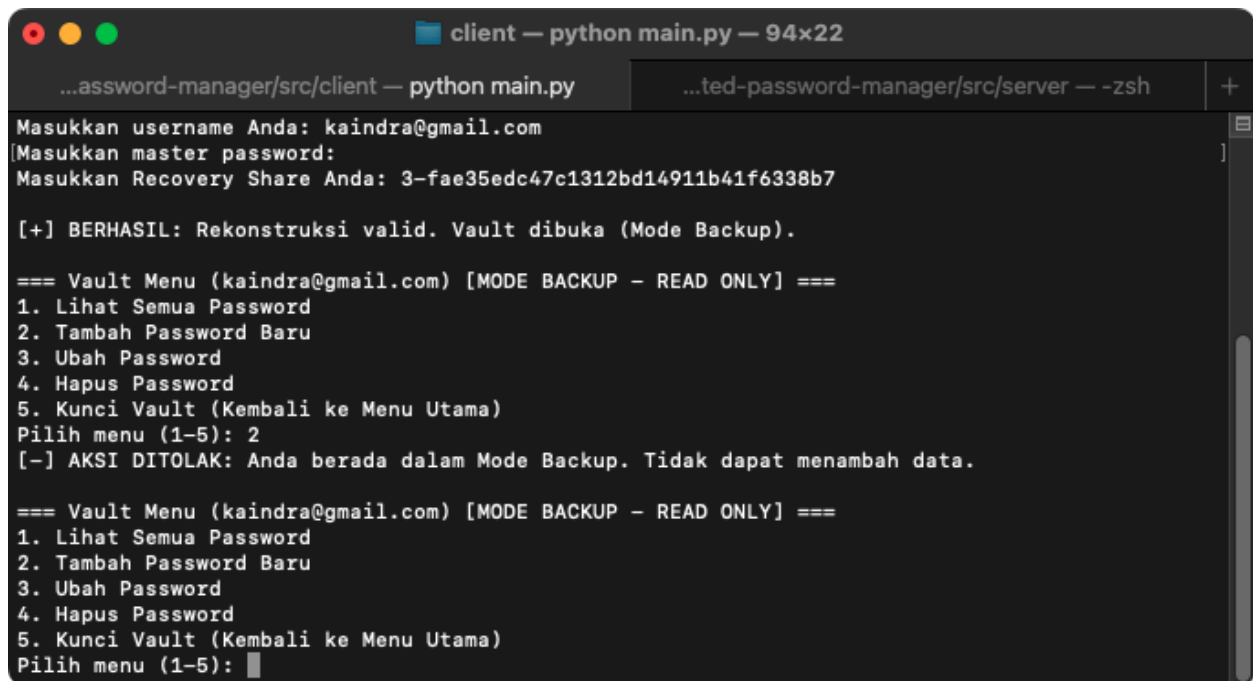
Akses vault secara luring terbukti berhasil karena fungsi `open_vault_backup` mengeksekusi proses dekripsi dengan data lokal dan *input* pengguna. Sistem menggunakan kunci turunan dari *input master password* untuk mendekripsi `local_share_cipher` di memori klien, lalu meneruskan *string* `local_share` bersama *string* `recovery_share` (dari input pengguna) ke fungsi `sss.reconstruct_key()`. Hasil perhitungan Shamir Secret Sharing dari kedua *share* ini mereproduksi susunan biner *master key*, yang diumpankan ke fungsi `aes_gcm.decrypt_data()` untuk mendekripsi `backup_vault_cipher` dari `client_data.json`.

3.7.2 Pembatasan Akses (*Read-Only*)

Komponen	Keterangan
Tujuan Uji	Memastikan implementasi <i>read-only</i> pada mode <i>backup</i> sudah dilakukan.
Input	Perintah tambah, ubah, atau hapus pada CLI.
Prosedur	Setelah berhasil masuk ke mode <i>backup</i> , eksekusi menu untuk menambah, mengubah, atau menghapus data <i>password</i> .
Output yang Diharapkan	Aplikasi memblokir aksi selain <i>read</i> .

a. Hasil

Tampilan Terminal Client



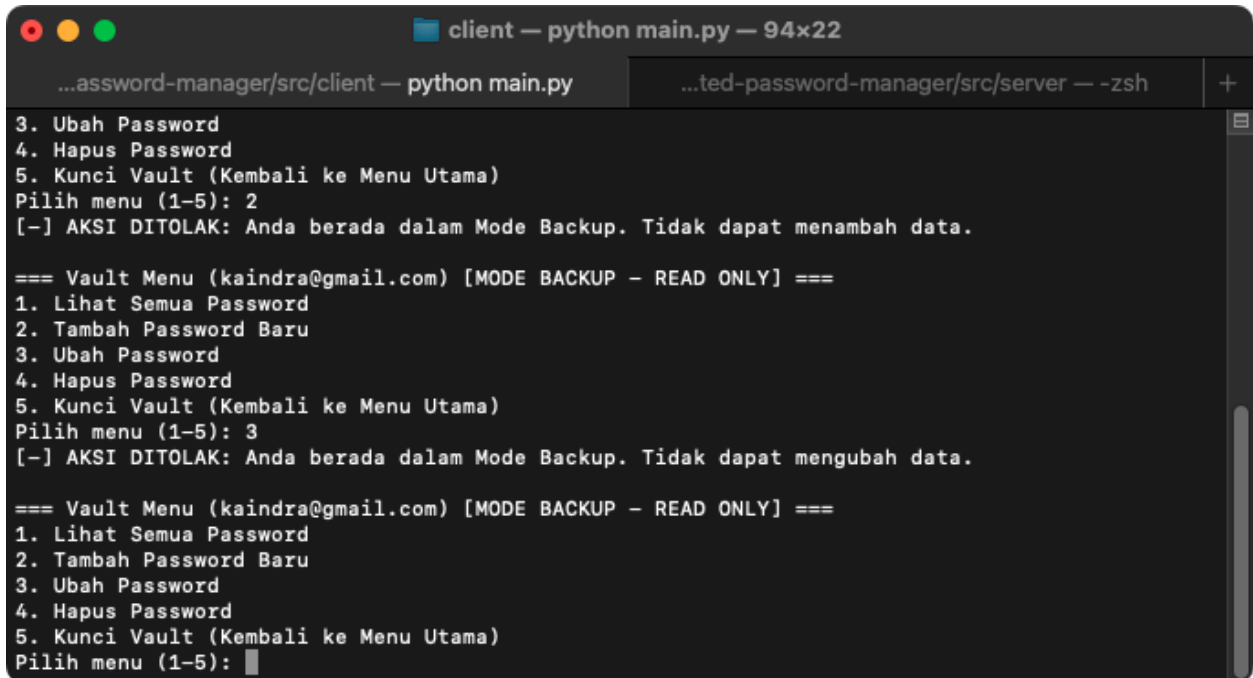
```
client — python main.py — 94x22
...assword-manager/src/client — python main.py  ...ted-password-manager/src/server — -zsh +
Masukkan username Anda: kaindra@gmail.com
[Masukkan master password:
Masukkan Recovery Share Anda: 3-fae35edc47c1312bd14911b41f6338b7

[+] BERHASIL: Rekonstruksi valid. Vault dibuka (Mode Backup).

=== Vault Menu (kaindra@gmail.com) [MODE BACKUP - READ ONLY] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5): 2
[-] AKSI DITOLAK: Anda berada dalam Mode Backup. Tidak dapat menambah data.

=== Vault Menu (kaindra@gmail.com) [MODE BACKUP - READ ONLY] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5):
```

Gambar 3.7.2 Penambahan *password* gagal akibat *read-only*



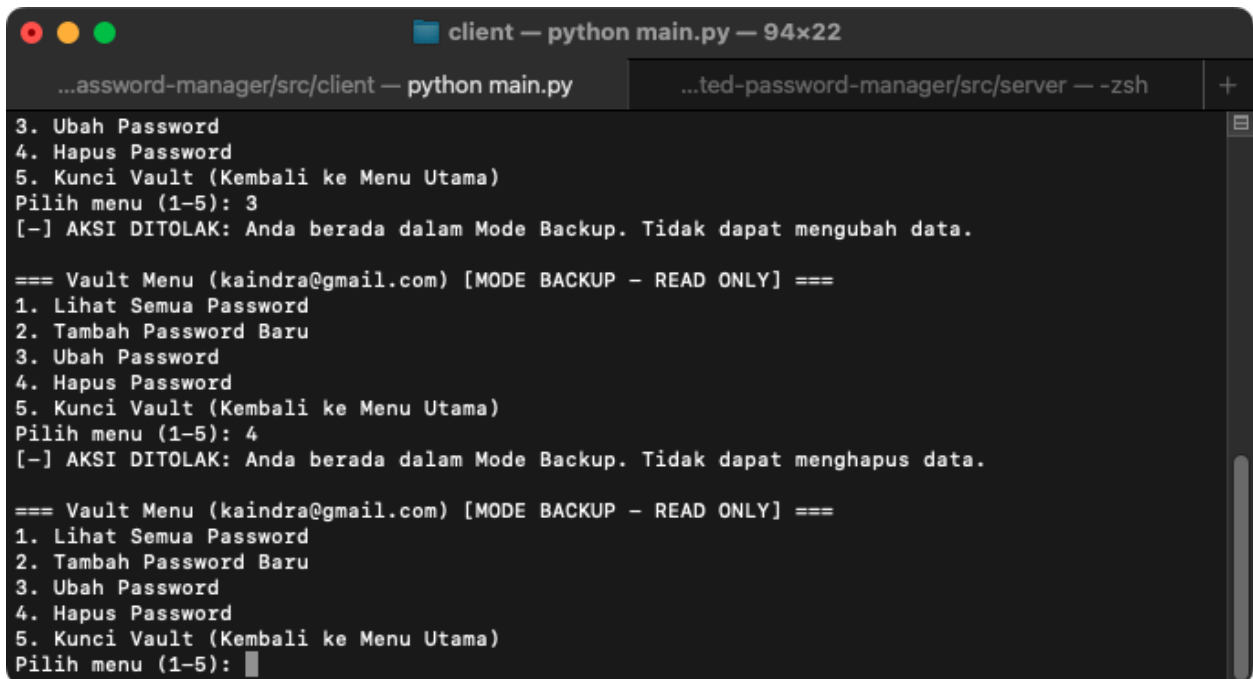
A terminal window titled "client — python main.py — 94x22" with tabs for "...assword-manager/src/client — python main.py" and "...ted-password-manager/src/server — -zsh". The output shows a menu with options 3 (Ubah Password), 4 (Hapus Password), and 5 (Kunci Vault). Option 3 is selected, leading to a "Vault Menu" for "kaindra@gmail.com" in "MODE BACKUP - READ ONLY". This menu also has options 1-5. Option 3 is selected again, but the user is notified: "[~] AKSI DITOLAK: Anda berada dalam Mode Backup. Tidak dapat menambah data." The process repeats with option 4 selected, resulting in another error: "[~] AKSI DITOLAK: Anda berada dalam Mode Backup. Tidak dapat mengubah data." The terminal ends with the menu displayed again and option 4 selected.

```
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5): 2
[-] AKSI DITOLAK: Anda berada dalam Mode Backup. Tidak dapat menambah data.

=== Vault Menu (kaindra@gmail.com) [MODE BACKUP - READ ONLY] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5): 3
[-] AKSI DITOLAK: Anda berada dalam Mode Backup. Tidak dapat mengubah data.

=== Vault Menu (kaindra@gmail.com) [MODE BACKUP - READ ONLY] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5):
```

Gambar 3.7.3 Pengubahan *password* gagal akibat *read-only*



A terminal window titled "client — python main.py — 94x22" with tabs for "...assword-manager/src/client — python main.py" and "...ted-password-manager/src/server — -zsh". The output shows the same menu as in Gambar 3.7.3. Option 3 is selected, leading to the "Vault Menu" in "MODE BACKUP - READ ONLY". Option 4 is selected, resulting in an error: "[~] AKSI DITOLAK: Anda berada dalam Mode Backup. Tidak dapat menghapus data." The terminal ends with the menu displayed again and option 4 selected.

```
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5): 3
[-] AKSI DITOLAK: Anda berada dalam Mode Backup. Tidak dapat mengubah data.

=== Vault Menu (kaindra@gmail.com) [MODE BACKUP - READ ONLY] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5): 4
[-] AKSI DITOLAK: Anda berada dalam Mode Backup. Tidak dapat menghapus data.

=== Vault Menu (kaindra@gmail.com) [MODE BACKUP - READ ONLY] ===
1. Lihat Semua Password
2. Tambah Password Baru
3. Ubah Password
4. Hapus Password
5. Kunci Vault (Kembali ke Menu Utama)
Pilih menu (1-5):
```

Gambar 3.7.4 Penghapusan *password* gagal akibat *read-only*

b. Analisis

Implementasi *read-only* pada mode luring berhasil melalui mekanisme pada `main.py`. Setelah *vault* berhasil didekripsi melalui mode darurat, `main` secara melempar argumen `is_backup=True` saat memanggil fungsi `vault_menu()`. Nilai ini kemudian dievaluasi oleh blok `if is_backup` pada setiap pilihan modifikasi data (penambahan, pengubahan, dan penghapusan). Kondisi `true` pada blok tersebut menghentikan alur eksekusi yang menghentikan eksekusi dan mencetak pesan penolakan ke CLI.

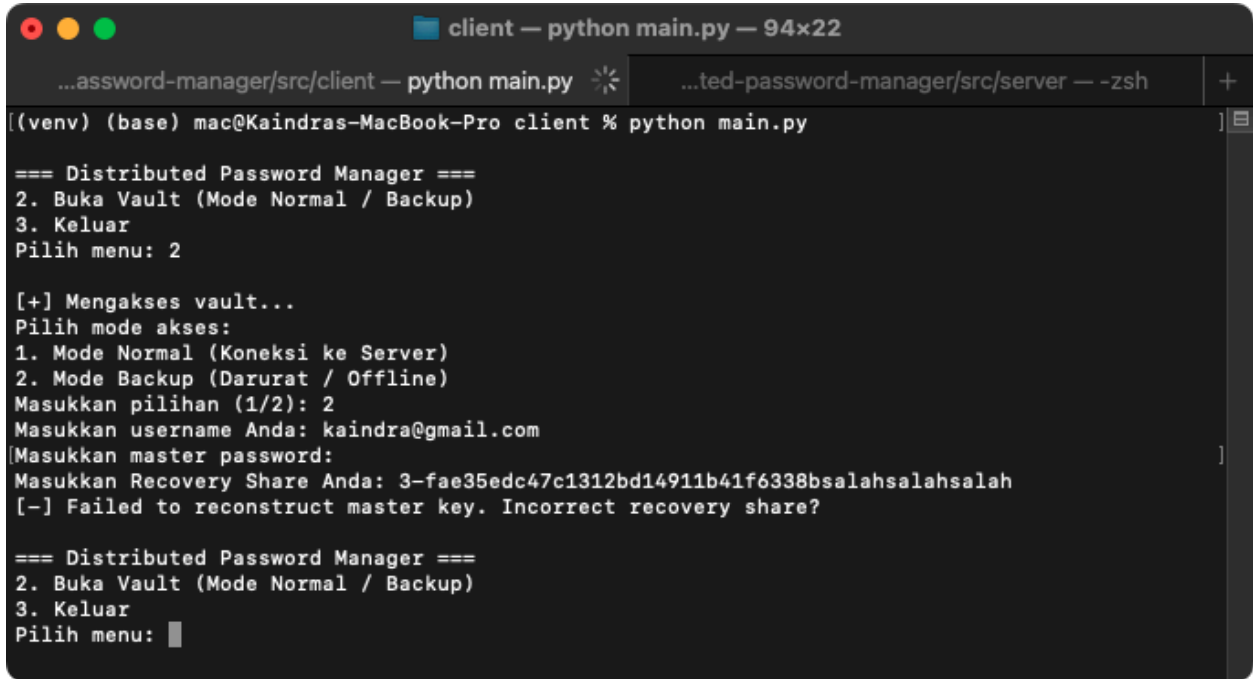
3.8 Pengujian Kegagalan Pemulihan

3.8.1 Penggunaan *Recovery Share* yang Salah

Komponen	Keterangan
Tujuan Uji	Memastikan penolakan akses apabila kunci cadangan tidak cocok.
Input	<code>master_password</code> yang valid dan nilai <code>recovery_share</code> yang salah.
Prosedur	Masuk ke menu akses mode <i>backup</i> , masukkan kata sandi yang benar, namun berikan string <i>recovery share</i> yang salah.
Output yang Diharapkan	Algoritma Shamir Secret Sharing gagal melakukan rekonstruksi <i>master key</i> yang valid. Akibatnya, proses dekripsi <i>vault</i> gagal, dan sistem dipastikan tidak menampilkan data <i>password</i> apa pun.

a. Hasil

Tampilan Terminal Client



```
client — python main.py — 94x22
...assword-manager/src/client — python main.py
...ted-password-manager/src/server — -zsh

[(venv) (base) mac@Kaindras-MacBook-Pro client % python main.py]

=== Distributed Password Manager ===
2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: 2

[+] Mengakses vault...
Pilih mode akses:
1. Mode Normal (Koneksi ke Server)
2. Mode Backup (Darurat / Offline)
Masukkan pilihan (1/2): 2
Masukkan username Anda: kaindra@gmail.com
[Masukkan master password:
Masukkan Recovery Share Anda: 3-fae35edc47c1312bd14911b41f6338bsalahsalahsalah
[-] Failed to reconstruct master key. Incorrect recovery share?

=== Distributed Password Manager ===
2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: █
```

Gambar 3.8.1 Gagal masuk karena *recovery share* salah

b. Analisis

Penolakan akses saat recovery share salah berhasil diuji. Ketika CLI meneruskan *string recovery share* bersama *local share* ke fungsi `sss.reconstruct_key()`, algoritma mereproduksi susunan biner *master key* yang cacat. Saat *master key* ini diteruskan ke `aes_gcm.decrypt_data()`, mekanisme AES-128-GCM mendeteksi ketidaksesuaian melalui kegagalan pada validasi tag autentikasi. Kegagalan fungsi ini memicu *exception*, yang ditangkap oleh blok di klien untuk menghentikan alur pemuatan data, sehingga sistem tetap terkunci.

3.8.2 Manipulasi *Backup Vault*

Komponen	Keterangan
Tujuan Uji	Memverifikasi fungsionalitas validasi tag autentikasi AES-128-GCM terhadap berkas lokal.
Input	Berkas lokal yang sudah dimodifikasi (nilai <code>backup_vault_cipher</code> pada <code>client_data.json</code> diubah).

Komponen	Keterangan
Prosedur	Lakukan perubahan nilai karakter pada <i>backup_vault</i> di penyimpanan lokal. Setelah itu, jalankan prosedur masuk mode <i>backup</i> dengan <i>master password</i> dan <i>recovery share</i> yang valid.
Output yang Diharapkan	Operasi dekripsi AES-128-GCM menggagalkan proses dan melempar <i>exception</i> karena validasi autentikasi tidak sesuai akibat modifikasi <i>ciphertext</i> . Sistem langsung menolak akses dan tidak menampilkan data <i>password</i> .

a. Hasil

Tampilan Terminal Client

```

client — python main.py — 94x22
...assword-manager/src/client — python main.py
...ted-password-manager/src/server — -zsh
+
[(venv) (base) mac@Kaindras-MacBook-Pro client % python main.py]
=== Distributed Password Manager ===
2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: 2

[+] Mengakses vault...
Pilih mode akses:
1. Mode Normal (Koneksi ke Server)
2. Mode Backup (Darurat / Offline)
Masukkan pilihan (1/2): 2
Masukkan username Anda: kaindra@gmail.com
Masukkan master password:
Masukkan Recovery Share Anda: 3-fae35edc47c1312bd14911b41f6338b7
[-] non-hexadecimal number found in fromhex() arg at position 36

=== Distributed Password Manager ===
2. Buka Vault (Mode Normal / Backup)
3. Keluar
Pilih menu: █

```

Gambar 3.8.2 *Vault* tidak bisa diakses karena *backup_vault_cipher* dimanipulasi

```

src > client > {} client_data.json > abc backup_vault_cipher
1 {
2   "kdf_salt": "cc5bc13240fa47585763469ad4589db8",
3   "local_share_cipher": "99f9b9b0e098caf2e6accba123bccbe5863fce5ae37afe178576b422a133b9fc6b83db3dd95d300cabd",
4   "local_share_nonce": "d5272acc3d10cb2ea811f4b3",
5   "backup_vault_cipher": "294390733fdcfb478ad4452b550bc6144391salah",
6   "backup_vault_nonce": "3ab551b5b29d06f88f307c2e"
7 }

```

Gambar 3.8.3 *backup_vault_cipher* diubah secara manual

b. Analisis

sistem berhasil mendeteksi dan menolak berkas *vault* cadangan yang dimodifikasi. Meskipun pengguna memberikan kredensial yang sah (fungsi `sss.reconstruct_key()` sukses memulihkan *master key*), eksekusi akan tetap gagal. Ketika *master key* diteruskan ke fungsi `aes_gcm.decrypt_data()`, AES-128-GCM memvalidasi kerusakan *ciphertext* melalui kegagalan verifikasi tag autentikasi. Hal ini memicu *exception* yang menghentikan konversi teks menjadi dictionary JSON, memastikan sistem memblokir akses terhadap data cadangan yang telah dimodifikasi.

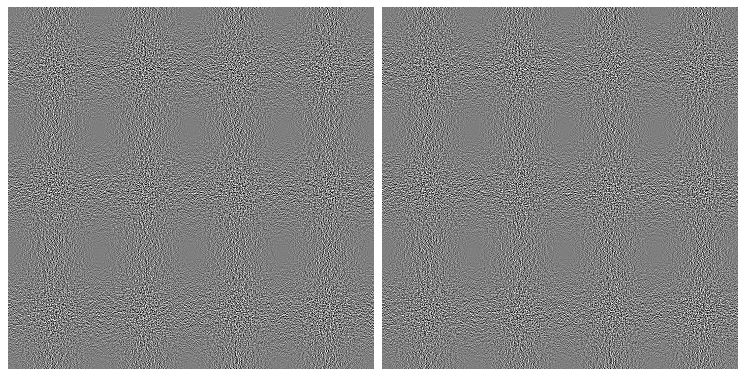
3.9 Pengujian Kriptografi Visual (*Bonus*)

3.9.1 Pembangkitan dan Pembagian Gambar *Share*

Komponen	Keterangan
Tujuan Uji	Memverifikasi skema pembagian, menghasilkan dua gambar independen yang menyembunyikan informasi asli.
Input	Nilai <code>recovery_share</code> yang dihasilkan sistem saat proses pembuatan <i>vault</i> .
Prosedur	Lakukan inisialisasi akun. Buka direktori <i>output</i> untuk memeriksa QR Code dan dua gambar <i>shares</i> .
Output yang Diharapkan	Sistem berhasil menciptakan dua gambar <i>share</i> . Masing-masing gambar secara visual hanya menampilkan pola acak.

c. Hasil

Tampilan *Share 1* dan *Share 2*



Gambar 3.9.1 QR berhasil dipecah menjadi dua gambar *share* (share_1.png dan share_2.png)

d. Analisis

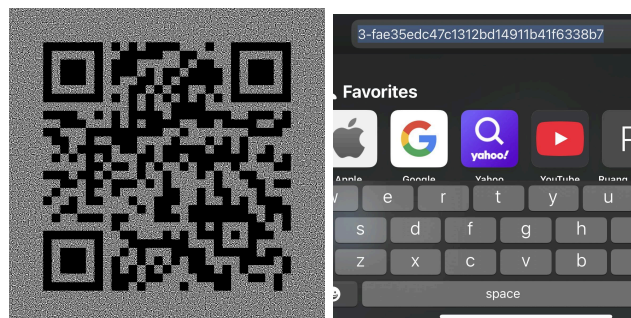
Penciptaan gambar *share* yang hanya berupa noise acak berhasil. Eksekusi diawali dengan konversi teks menjadi matriks QR Code dengan modul qrcode, yang setiap pikselnya diekspansi menjadi blok matriks 2x2 piksel. Program memilih satu dari enam pola dasar berasio warna 50:50 dan menetakannya pada blok Share 1, dilanjutkan dengan pemetaan berdasarkan piksel QR Code asli; algoritma memberikan pola yang identik pada blok Share 2 jika piksel asli putih, atau kebalikannya (*inverse*) jika piksel asli hitam.

3.9.2 Kombinasi Gambar *Share*

Komponen	Keterangan
Tujuan Uji	Memastikan penggabungan kedua <i>share</i> dapat merekonstruksi QR yang sah dan dapat dibaca.
Input	share1.png dan share2.png
Prosedur	Tampilkan gambar hasil penggabungan dari kedua gambar <i>share</i> . Pindai gambar dengan <i>smartphone</i> .
Output yang Diharapkan	Gambar hasil penggabungan memperlihatkan pola QR yang jelas, pemindai berhasil membaca QR dan menghasilkan teks <i>recovery share</i> yang sama persis dengan yang diciptakan saat pembuatan akun.

e. Hasil

Tampilan QR Hasil Kombinasi



Gambar 3.9.2 QR awal berhasil direkonstruksi ulang dari kedua gambar *share* dan berhasil dipindai menghasilkan *recovery share* yang sama

f. Analisis

Pemulihan QR Code melalui superposisi berhasil diuji. Fungsi memanggil operasi komparasi `min()` terhadap koordinat yang sama pada Share 1 dan Share 2. Hal ini menyebabkan dua blok berpola identik (putih) mengalami tumpang tindih dan mempertahankan rasio piksel 50:50 (terlihat sebagai abu-abu visual). Sebaliknya, pada blok berpola kebalikan (hitam), nilai piksel hitam (0) saling melengkapi piksel putih (255), menutupi blok 2×2 menjadi hitam. Hal ini memastikan pola QR Code kembali utuh dan valid meskipun latarnya menjadi abu-abu.

BAB IV

KESIMPULAN

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat ditarik beberapa kesimpulan utama sebagai berikut.

1. Arsitektur *client-server* yang dibangun terbukti berhasil memenuhi prinsip *zero-knowledge* secara ketat. Klien memegang kendali penuh atas seluruh proses kriptografi, sementara *server* hanya bertindak sebagai media penyimpanan untuk data *ciphertext* bertipe BLOB, *nonce*, dan *server share*.
2. Integrasi berbagai algoritma kriptografi modern beroperasi secara presisi. Shamir Secret Sharing dengan skema (2,3) berhasil mendistribusikan dan merekonstruksi *master key* dengan syarat ambang batas yang sesuai. Selanjutnya, algoritma AES-128-GCM terbukti tangguh dalam menjaga kerahasiaan sekaligus integritas *vault*. Segala bentuk modifikasi ilegal pada penyimpanan atau penggunaan kunci yang salah akan langsung ditolak sistem karena kegagalan verifikasi *tag* autentikasi Galois/Counter Mode.
3. Sistem mampu menangani dua mode akses dengan baik. Pada *mode Normal*, setiap operasi manipulasi data (*create*, *update*, *delete*) berhasil dienkripsi ulang secara utuh dengan *nonce* yang selalu diperbarui, lalu disinkronisasikan ke *server* dan *backup* lokal. Pada *mode Backup*, sistem secara efektif menggunakan *local share* dan *recovery share* untuk memulihkan *vault* luring, dan sukses menerapkan kebijakan *read-only* untuk mencegah terjadinya konflik sinkronisasi data dengan *server*.
4. Konversi *recovery share* menjadi QR Code yang kemudian dipecah menggunakan algoritma kriptografi visual berjalan sesuai teori. Sistem berhasil mendistribusikan QR *code* ke dalam dua gambar *share* biner yang secara visual hanya berupa *noise* acak tak bermakna. Proses superposisi logika AND (menggunakan komparasi nilai minimum piksel) terhadap kedua gambar tersebut mampu memulihkan pola QR Code hingga dapat dipindai kembali dengan sempurna.

Secara keseluruhan, aplikasi telah memenuhi seluruh spesifikasi dan kriteria yang ditetapkan.

DAFTAR PUSTAKA

- Munir, Rinaldi (2024). Advanced Encryption Standard (AES). [Online] Tersedia: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi-dan-Koding/2023-2024/08-AES-2024> [Diakses 23 Mei 2026]
- Munir, Rinaldi (2026). Skema Pembagian Data Rahasia. [Online] Tersedia: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi-dan-Koding/2025-2026/36-Skema-Pembagian-Data-Rahasia-2026.pdf> [Diakses 23 Mei 2026]
- Munir, Rinaldi (2026). Kriptografi Visual, Teori dan Aplikasinya. [Online] Tersedia: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi-dan-Koding/2025-2026/37-Kriptografi-Visual-Bagian1-2026.pdf> [Diakses 23 Mei 2026]
- Wikipedia (2026). Cryptographically secure pseudorandom number generator. [Online] Tersedia: https://en.wikipedia.org/wiki/Cryptographically_secure_pseudorandom_number_generator [Diakses 23 Mei 2026]
- Wikipedia (2026). NIST SP 800-90A. [Online] Tersedia: https://en.wikipedia.org/wiki/NIST_SP_800-90A [Diakses 23 Mei 2026]
- National Institute of Standards and Technology (2010). NIST SP 800-132: Recommendation for Password-Based Key Derivation. [Online] Tersedia: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-132.pdf> [Diakses 23 Mei 2026]
- OWASP (2026). Password Storage Cheat Sheet. [Online] Tersedia: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html [Diakses 23 Mei 2026]

LAMPIRAN

Repository Github:

<https://github.com/0xNathaniel/distributed-password-manager>

Video Demo: <https://youtu.be/m6PlhX3shw4>

Pembagian Tugas:

Tugas		PIC
1.	Inisialisasi <i>environment</i> , konfigurasi <i>repository</i> , dan <i>containerization</i> (Docker), merancang basis data SQLite serta API <i>endpoints</i> Fast API berbasis <i>zero-knowledge</i> .	Nathaniel Jonathan Rusli (13523013)
2.	Membangun CLI dan struktur data <i>vault (client)</i> untuk operasi CRUD, mengimplementasikan pembangkit <i>password</i> (CSPRNG), <i>logic mode</i> Normal dan Backup, serta fitur bonus Kriptografi Visual.	Maheswara Bayu Kaindra (13523015)
3.	Mengimplementasikan kriptografi inti di sisi <i>client</i> : Shamir Secret Sharing (2,3), penurunan kunci dengan Key Derivation Function, dan proses enkripsi/dekripsi <i>vault</i> menggunakan AES-128-GCM.	Peter Wongsoredjo (13523039)